

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA KỸ THUẬT ĐIỆN TỬ 1



BÁO CÁO CUỐI KÌ

HỌC SÂU

Giảng viên: TS. NGUYỄN QUỐC UY

Họ và tên SV: Nguyễn Hữu Hoàng Anh - B23DCDK007

Phạm Tuấn Anh - B23DCDK012

Nguyễn Hoàng Anh - B23DCDK006

Lớp: D23CQDKRB

Hà Nội, 27/5/2025

LỜI CẢM ƠN

Em xin gửi lời cảm ơn chân thành và sâu sắc nhất đến Thầy **TS. Nguyễn Quốc Uy**, giảng viên giảng dạy học phần *Học Sâu*, Khoa Kỹ Thuật Điện Tử 1, Học viện Công nghệ Bưu chính Viễn thông, người đã tận tâm truyền đạt những kiến thức nền tảng và hướng dẫn em trong suốt quá trình học tập cũng như thực hiện báo cáo này.

Những kiến thức mà Thầy cung cấp không chỉ giúp em hiểu rõ bản chất của các mô hình học máy, từ mạng nơ-ron truyền thống, CNN, RNN, LSTM đến các kiến trúc tiên tiến như Transformer, mà còn tạo nền tảng vững chắc để em có thể triển khai thực tế một hệ thống AI phức tạp kết hợp cả Thị giác máy tính và Xử lý ngôn ngữ tự nhiên. Sự tận tâm của Thầy trong từng bài giảng và bài tập thực hành là động lực lớn giúp em hoàn thành tốt đồ án cuối kỳ này.

Em đã cố gắng hoàn thiện báo cáo một cách tốt nhất có thể, tuy nhiên với khối lượng kiến thức đồ sộ của dự án và kinh nghiệm cá nhân còn hạn chế, bài làm khó tránh khỏi những thiếu sót. Em rất mong nhận được những ý kiến đóng góp quý báu của Thầy để có thể rút kinh nghiệm và hoàn thiện hơn trong tương lai.

Cuối cùng, em xin kính chúc Thầy **TS. Nguyễn Quốc Uy** cùng quý thầy cô trong Khoa Kỹ Thuật Điện Tử 1 dồi dào sức khỏe, hạnh phúc và luôn thành công trong sự nghiệp trồng người.

MỤC LỤC

LỜI CẢM ƠN	i
DANH MỤC HÌNH ẢNH	vi
DANH MỤC BẢNG BIỂU	vii
DANH MỤC KÝ HIỆU VÀ CHỮ VIẾT TẮT	viii
LỜI MỞ ĐẦU	ix
1 TỔNG QUAN VÀ LÝ DO CHỌN ĐỀ TÀI	1
1.1 Bối cảnh và lý do chọn đề tài	1
1.2 Phát biểu bài toán	1
1.3 Mục tiêu nghiên cứu và phát triển	2
1.4 Phạm vi nghiên cứu	2
1.5 Phương pháp thực hiện	3
1.6 Cấu trúc báo cáo	4
2 KIẾN TRÚC HỆ THỐNG VÀ KIẾN TRÚC CÁC MÔ HÌNH	5
2.1 Kiến trúc tổng thể hệ thống	5
2.2 Luồng dữ liệu từ đầu vào đến đầu ra	5
2.2.1 Các bước xử lý chính	5
2.3 Mô hình nhận diện ký hiệu ở tầng thị giác	6
2.3.1 Phát biểu bài toán	7
2.3.2 Vai trò của MediaPipe Holistic trong pipeline	7
2.3.3 Biểu diễn đầu vào theo chuỗi landmarks	8
2.3.4 Kiến trúc mô hình TFLite (Kaggle ASL Signs 1st Place)	9
2.3.5 Nguồn gốc mô hình và dữ liệu huấn luyện	10
2.3.6 Cơ chế suy luận thời gian thực	11
2.3.7 Ưu điểm và giới hạn của thiết kế hiện tại	11

2.4	Mô hình ghép từ thành câu tiếng Việt	12
2.4.1	Phát biểu bài toán	13
2.4.2	Kiến trúc lai được sử dụng	13
2.4.3	Lý do chọn BARTpho	15
2.4.4	Thông số thiết kế chính	16
2.4.5	Ưu điểm và giới hạn	17
2.5	Mô hình dịch Việt - Lào	17
2.5.1	Bài toán dịch máy trong phạm vi đề tài	18
2.5.2	Lý do chọn NLLB-200 Distilled 600M	18
2.5.3	Chiến lược tinh chỉnh bằng LoRA	19
2.5.4	Tokenizer và cơ chế sinh câu đích	20
2.5.5	Ưu điểm và những rủi ro cần lưu ý	20
2.6	Cơ chế tích hợp giữa ba mô hình	21
2.7	Dữ liệu huấn luyện và tiền xử lý	22
2.7.1	Mô hình pre-trained cho mô-đun nhận diện ký hiệu	22
2.7.2	Bộ dữ liệu cho mô-đun ghép câu tiếng Việt	23
2.7.3	Tiền xử lý cho mô-đun ghép câu	24
2.7.4	Bộ dữ liệu cho mô-đun dịch Việt - Lào	25
2.7.5	Tiền xử lý cho mô-đun dịch Việt - Lào	26
3	XÂY DỰNG PHẦN MỀM VÀ TÍCH HỢP HỆ THỐNG	27
3.1	Yêu cầu chức năng và phi chức năng	27
3.1.1	Yêu cầu chức năng	27
3.1.2	Yêu cầu phi chức năng	27
3.2	Phân tích use case	27
3.3	Xây dựng mô-đun nhận diện ký hiệu	28
3.3.1	Cấu trúc mã nguồn của mô-đun	29
3.3.2	Hiện thực trích xuất landmarks bằng MediaPipe	29
3.3.3	Hiện thực nạp và suy luận mô hình TFLite	30
3.3.4	Hiện thực suy luận thời gian thực	31

3.3.5	Nhận xét dưới góc nhìn kỹ thuật phần mềm	32
3.4	Xây dựng mô-đun ghép từ thành câu tiếng Việt	33
3.4.1	Các thành phần chính trong mã nguồn	33
3.4.2	Luồng xử lý chính	34
3.4.3	Hiện thực tầng rule-based và fallback	35
3.4.4	Hiện thực suy luận bằng BARTpho	36
3.4.5	Khả năng tích hợp với pipeline realtime	37
3.5	Xây dựng mô-đun dịch Việt - Lào	37
3.5.1	Các thành phần chính trong mã nguồn	37
3.5.2	Quy trình khởi tạo mô hình	38
3.5.3	Quy trình suy luận một câu dịch	38
3.5.4	Vai trò của LoRA trong phần mềm	39
3.5.5	Tích hợp với mô-đun ghép câu phía trước	40
3.5.6	Nhận xét về hiện thực phần mềm	40
3.6	Tích hợp pipeline thời gian thực	40
3.7	Giao diện chạy thử và các đoạn mã tiêu biểu	41
4	THỰC NGHIỆM, ĐÁNH GIÁ, KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	42
4.1	Thiết lập thực nghiệm	42
4.2	Đánh giá mô hình nhận diện ký hiệu	43
4.2.1	Các metric có thể đánh giá	43
4.2.2	Bối cảnh mô hình pre-trained	44
4.2.3	Công cụ đánh giá định lượng	45
4.2.4	Đánh giá thực nghiệm qua demo realtime	46
4.2.5	Đánh giá định tính và phân tích lỗi	47
4.2.6	Lưu ý khi diễn giải kết quả	47
4.3	Đánh giá mô hình khôi phục câu tiếng Việt	48
4.3.1	Ý nghĩa của từng nhóm metric	49
4.3.2	Kết quả thực nghiệm trên tập test	49
4.3.3	Phân tích kết quả	51

4.3.4	Đánh giá định tính và rủi ro diễn giải	52
4.4	Đánh giá mô hình dịch Việt - Lào	53
4.4.1	Các metric nên sử dụng	53
4.4.2	Giao thức đánh giá nên áp dụng cho project	54
4.4.3	Nhận xét dựa trên trạng thái repository hiện tại	55
4.4.4	Đánh giá định tính nên nhấn mạnh điều gì	56
4.4.5	Ý nghĩa của đánh giá đối với toàn hệ thống	56
4.5	Đánh giá tích hợp toàn hệ thống	56
4.5.1	Nội dung nên đánh giá	57
4.6	Kết luận đề tài	58
4.7	Hạn chế hiện tại	58
4.8	Hướng phát triển	59
TÀI LIỆU THAM KHẢO		60

DANH MỤC HÌNH ẢNH

Hình 2.1	Vị trí của mô-đun nhận diện thị giác trong pipeline tổng thể	7
Hình 2.2	Minh họa landmarks pose, khuôn mặt và bàn tay được trích xuất bởi MediaPipe Holistic	8
Hình 2.3	Sơ đồ biến video thành chuỗi landmarks kích thước $N \times 543 \times 3$.	9
Hình 2.4	Vị trí của mô-đun ghép câu trong hệ thống xử lý ký hiệu	13
Hình 2.5	Pipeline xử lý từ gloss thô đến câu tiếng Việt hoàn chỉnh	14
Hình 2.6	Kiến trúc các thành phần chính của mô-đun ghép câu	14
Hình 2.7	Kiến trúc sinh câu tiếng Việt bằng BARTpho	16
Hình 2.8	Quy trình huấn luyện và đánh giá BARTpho	17
Hình 2.9	Vị trí của mô-đun dịch Việt - Lào trong pipeline tổng thể	18
Hình 2.10	Kiến trúc NLLB-200 kết hợp adapter LoRA trong mô-đun dịch Việt - Lào	20
Hình 2.11	Cấu trúc một mẫu dữ liệu gloss-to-sentence	23
Hình 2.12	Số lượng mẫu trong các tập train, validation và test	24
Hình 2.13	Tỷ lệ chia dữ liệu train, validation và test	24
Hình 2.14	Tỉ lệ chia bộ dữ liệu song ngữ Việt – Lào	26
Hình 3.1	Sơ đồ sliding window 16–60 frame và logic “chỉ gom khi có tay” trong suy luận realtime	32
Hình 3.2	Các file báo cáo được sinh sau khi đánh giá mô hình	34
Hình 3.3	Lưu đồ xử lý chuỗi gloss trong mô-đun ghép câu	35
Hình 3.4	Cơ chế trạng thái của SentenceBuffer khi thêm, xoá và tạo câu .	36
Hình 3.5	Kiểm thử suy luận BARTpho bằng chuỗi gloss đầu vào	36
Hình 3.6	Luồng cài đặt và suy luận trong mô-đun dịch Việt – Lào	39
Hình 4.1	Nhóm ý nghĩa của các metric đánh giá sinh văn bản	49
Hình 4.2	Kết quả đánh giá tự động trên toàn bộ tập test	50
Hình 4.3	Biểu đồ các metric đánh giá BARTpho trên tập test	51
Hình 4.4	Biểu đồ Test Loss và Perplexity của BARTpho	51
Hình 4.5	Sự thay đổi BLEU của mô-đun dịch Việt – Lào theo từng epoch .	55

DANH MỤC BẢNG BIỂU

Bảng 2.1	Thành phần vector đặc trưng ở mỗi frame	8
Bảng 2.2	Thông số chính của mô hình nhận diện	10
Bảng 2.3	Vai trò của các thành phần trong mô-đun ghép câu	15
Bảng 2.4	Cấu hình fine-tune BARTpho	16
Bảng 2.5	Thông tin mô hình pre-trained cho mô-đun nhận diện ký hiệu . . .	22
Bảng 2.6	Thống kê bộ dữ liệu VSL Vietnamese NLP Dataset v4 Mega . . .	23
Bảng 2.7	Thống kê dữ liệu song ngữ Việt - Lào trong repository	25
Bảng 3.1	Các file chính của mô-đun nhận diện	29
Bảng 3.2	Vai trò của các file quan trọng trong mô-đun ghép câu	33
Bảng 3.3	Các thành phần phần mềm chính của mô-đun dịch Việt - Lào . . .	38
Bảng 4.1	Môi trường và dữ liệu dùng trong thực nghiệm	42
Bảng 4.2	Ý nghĩa của các metric cho mô-đun nhận diện	44
Bảng 4.3	Thông tin tham chiếu từ cuộc thi Kaggle ASL Signs	45
Bảng 4.4	Kết quả benchmark suy luận TFLite của mô-đun nhận diện	45
Bảng 4.5	Trạng thái các metric phân loại của mô-đun nhận diện trong snap- shot hiện tại	46
Bảng 4.6	Đánh giá định tính mô-đun nhận diện qua webcam	47
Bảng 4.7	Ý nghĩa của các metric dùng cho mô-đun ghép câu tiếng Việt . . .	49
Bảng 4.8	Kết quả đánh giá mô-đun ghép câu tiếng Việt trên tập test	50
Bảng 4.9	Một số mẫu lỗi của mô-đun ghép câu tiếng Việt	52
Bảng 4.10	Ý nghĩa của các metric đánh giá mô-đun dịch Việt - Lào	54
Bảng 4.11	Kết quả đánh giá định lượng mô-đun dịch Việt – Lào trích từ vilo.pdf	54
Bảng 4.12	Đánh giá định tính pipeline end-to-end	57

DANH MỤC KÝ HIỆU VÀ CHỮ VIẾT TẮT

Ký hiệu	Ý nghĩa
VSL	Vietnamese Sign Language (Ngôn ngữ Ký hiệu Việt Nam).
AI	Artificial Intelligence (Trí tuệ Nhân tạo).
NLP	Natural Language Processing (Xử lý Ngôn ngữ Tự nhiên).
CV	Computer Vision (Thị giác máy tính).
ASL	American Sign Language (Ngôn ngữ ký hiệu Hoa Kỳ).
WLASL	Word-Level American Sign Language dataset.
RGB	Red-Green-Blue, biểu diễn màu chuẩn của ảnh số.
TFLite	TensorFlow Lite, engine suy luận mô hình nhẹ của Google.
GRU	Gated Recurrent Unit, một biến thể mạng hồi quy có cổng.
TGCN	Temporal Graph Convolutional Network.
LoRA	Low-Rank Adaptation (Kỹ thuật tinh chỉnh tham số hiệu quả).
Keypoints	Tọa độ các điểm neo (khớp, mắt, mũi...) trên cơ thể.
MediaPipe	Thư viện mã nguồn mở của Google dùng để trích xuất 543 landmarks từ video.
BARTpho	Mô hình ngôn ngữ tạo sinh (Transformer) dành riêng cho tiếng Việt.
NLLB	No Language Left Behind, họ mô hình dịch đa ngôn ngữ của Meta AI.
BLEU	Bilingual Evaluation Understudy, metric đánh giá dịch máy theo n-gram.
ROUGE	Recall-Oriented Understudy for Gisting Evaluation, metric so khớp chuỗi tham chiếu.
chrF	Character n-gram F-score, metric đánh giá ở cấp ký tự.
TER	Translation Edit Rate, metric đo số phép chỉnh sửa cần thiết.
FPS	Frames Per Second, số khung hình xử lý được trong một giây.
ROC-AUC	Area Under the Receiver Operating Characteristic Curve.

LỜI MỞ ĐẦU

Trong bối cảnh trí tuệ nhân tạo và học máy ngày càng phát triển, công nghệ nhận dạng hành động và xử lý ngôn ngữ tự nhiên (NLP) đang đóng vai trò vô cùng quan trọng trong việc thu hẹp khoảng cách giao tiếp giữa con người và máy tính. Đặc biệt, đối với cộng đồng người khiếm thính, việc xây dựng một hệ thống có khả năng nhận dạng và dịch thuật Ngôn ngữ Ký hiệu (Sign Language) sang ngôn ngữ nói/viết là một bài toán cấp thiết và mang ý nghĩa nhân văn sâu sắc. Hơn thế nữa, trong kỷ nguyên hội nhập quốc tế, việc mở rộng khả năng dịch thuật đa ngôn ngữ (chẳng hạn như từ Tiếng Việt sang Tiếng Lào) sẽ càng phá vỡ rào cản ngôn ngữ, tạo ra một hệ sinh thái giao tiếp toàn diện.

Học phần *Học Sâu (Deep Learning)* trang bị cho sinh viên các kiến thức nền tảng và chuyên sâu về mạng nơ-ron nhân tạo, các cấu trúc mạng tiên tiến như CNN, RNN, LSTM, Transformer, cũng như các kỹ thuật tinh chỉnh mô hình lớn (Fine-tuning, LoRA). Thông qua các nội dung này, sinh viên có khả năng áp dụng các mô hình học sâu vào các bài toán phức tạp kết hợp đa luồng xử lý như Thị giác máy tính (Computer Vision) và Xử lý ngôn ngữ tự nhiên (NLP).

Báo cáo này trình bày quá trình nghiên cứu và phát triển một hệ thống dịch thuật ngôn ngữ ký hiệu toàn diện, bao gồm ba mô-đun AI cốt lõi nối tiếp nhau:

- 1. Nhận dạng ký hiệu ở tầng thị giác (Visual Sign Recognition):** Ứng dụng thư viện MediaPipe Holistic để trích xuất toàn bộ 543 landmarks (khuôn mặt, thân, hai bàn tay) từ video người dùng, kết hợp với mô hình TFLite pre-trained từ giải pháp đạt giải nhất cuộc thi Kaggle ASL Signs để dự đoán ra các gloss ở mức từ. Mô-đun này nhận diện 250 từ vựng ký hiệu ASL, sau đó ánh xạ sang gloss trung gian tiếng Việt để nối với các tầng NLP phía sau.
- 2. Khôi phục ngữ pháp Tiếng Việt (VSL Vietnamese NLP):** Vì ngôn ngữ ký hiệu thường bị thiếu cấu trúc ngữ pháp chuẩn, mô-đun này sử dụng mô hình ngôn ngữ BARTpho (kết hợp các quy tắc rules/templates) để khôi phục các từ vựng rời rạc thành một câu Tiếng Việt hoàn chỉnh và có nghĩa.
- 3. Dịch thuật Tiếng Việt - Tiếng Lào (Machine Translation):** Sử dụng mô hình *facebook/nllb-200-distilled-600M* được tinh chỉnh bằng kỹ thuật LoRA trên tập dữ liệu song ngữ Việt - Lào để dịch câu Tiếng Việt vừa sinh ra sang Tiếng Lào, hoàn thiện chu trình giao tiếp đa quốc gia.

Thông qua quá trình nghiên cứu và thực hiện đồ án, em có cơ hội hệ thống hóa lại các kiến thức về luồng xử lý dữ liệu phức tạp (Complex Data Pipeline). Việc tích hợp mô hình nhận diện ký hiệu pre-trained (TFLite), tinh chỉnh (fine-tune) mô hình ngôn ngữ đa

dịch thuật bằng LoRA trên môi trường máy học giúp em hiểu rõ hơn về cách các mô hình học sâu tương tác và truyền dữ liệu cho nhau trong thực tế. Đây là những kinh nghiệm vô cùng quý giá để phục vụ cho các dự án AI quy mô lớn trong tương lai.

Báo cáo được hoàn thành theo yêu cầu của học phần. Mặc dù em đã cố gắng thực hiện một cách đầy đủ và nghiêm túc, nhưng do khối lượng kiến thức đồ sộ và kinh nghiệm còn hạn chế nên bài làm khó tránh khỏi những thiếu sót. Em rất mong nhận được sự góp ý của Thầy để có thể hoàn thiện hơn trong các học phần tiếp theo.

Em xin chân thành cảm ơn.

Hà Nội, ngày 27 tháng 5 năm 2026

Sinh viên thực hiện

Nguyễn Hữu Hoàng Anh

1. TỔNG QUAN VÀ LÝ DO CHỌN ĐỀ TÀI

1.1 Bối cảnh và lý do chọn đề tài

Đề tài hướng đến bài toán hỗ trợ giao tiếp cho cộng đồng người khiếm thính thông qua việc nhận diện ngôn ngữ ký hiệu, khôi phục câu tiếng Việt hoàn chỉnh và tiếp tục dịch sang tiếng Lào. Đây là một bài toán có ý nghĩa xã hội rõ ràng, đồng thời là một minh họa tiêu biểu cho cách kết hợp thị giác máy tính, xử lý ngôn ngữ tự nhiên và dịch máy trong một pipeline học sâu thống nhất.

Trong giao tiếp hằng ngày, rào cản lớn nhất giữa người dùng ngôn ngữ ký hiệu và người không biết ký hiệu nằm ở việc thông tin thị giác không được chuyển đổi tức thời sang văn bản hoặc tiếng nói. Một hệ thống chỉ nhận diện từng cử chỉ đơn lẻ vẫn chưa đủ để giải quyết vấn đề này, vì đầu ra dạng nhãn rời rạc thường thiếu ngữ pháp, thiếu ngữ cảnh và khó sử dụng trực tiếp trong hội thoại. Do đó, đề tài lựa chọn hướng tiếp cận nhiều tầng: nhận diện ký hiệu ở tầng thị giác, chuẩn hóa chuỗi ký hiệu thành câu tiếng Việt ở tầng ngôn ngữ, sau đó dịch sang tiếng Lào để mở rộng khả năng giao tiếp liên ngôn ngữ.

Lý do chọn đề tài xuất phát từ ba khía cạnh chính. Thứ nhất, đề tài có tính nhân văn vì hướng đến hỗ trợ một nhóm người dùng có nhu cầu giao tiếp đặc thù. Thứ hai, đề tài có tính ứng dụng vì có thể triển khai thử nghiệm với webcam và các mô hình học sâu hiện có. Thứ ba, đề tài có tính liên ngành cao: mỗi tầng xử lý đều là một bài toán học máy riêng, nhưng giá trị cuối cùng chỉ xuất hiện khi các tầng được ghép nối thành một hệ thống vận hành được từ đầu vào video đến đầu ra văn bản.

1.2 Phát biểu bài toán

Bài toán trung tâm của đề tài là xây dựng một hệ thống có khả năng tiếp nhận chuỗi khung hình từ webcam, nhận diện các đơn vị ký hiệu, biến chuỗi từ thô thành câu tiếng Việt có ngữ pháp và tiếp tục dịch sang tiếng Lào. Như vậy, đầu vào của hệ thống là dữ liệu video theo thời gian thực, còn đầu ra là văn bản ở hai tầng: câu tiếng Việt đã được phục hồi và câu tiếng Lào tương ứng.

Về mặt hình thức, pipeline có thể được mô tả như sau:

Video → Landmarks → Gloss → Câu tiếng Việt → Câu tiếng Lào.

Ở bước đầu, video RGB được chuyển thành chuỗi landmarks bằng MediaPipe Holistic. Chuỗi landmarks sau đó được đưa vào mô hình nhận diện để dự đoán gloss hoặc nhãn từ vựng. Các gloss này chưa phải là câu tự nhiên, nên cần mô-đun NLP để bổ sung trật tự từ, dấu câu và các thành phần ngữ pháp. Cuối cùng, câu tiếng Việt được đưa vào mô hình dịch máy Việt – Lào.

Khó khăn của bài toán đến từ cả dữ liệu thị giác lẫn dữ liệu ngôn ngữ. Ở tầng thị giác, kết quả nhận diện chịu ảnh hưởng của góc quay, ánh sáng, tốc độ ký hiệu, che khuất bàn tay và sự khác biệt giữa từng người ký hiệu. Ở tầng ngôn ngữ, chuỗi gloss thường ngắn, thiếu hư từ và không tuân theo đầy đủ ngữ pháp tiếng Việt. Ở tầng dịch máy, mô hình phải bảo toàn đúng nghĩa của câu tiếng Việt trong khi tiếng Lào là ngôn ngữ ít tài nguyên hơn nhiều cặp dịch phổ biến. Ngoài ra, đây là bài toán đa giai đoạn, nên sai số của mô-đun trước có thể lan truyền và làm giảm chất lượng của mô-đun sau.

1.3 Mục tiêu nghiên cứu và phát triển

Mục tiêu của đề tài không chỉ dừng ở việc huấn luyện riêng lẻ từng mô hình mà còn hướng tới xây dựng được một hệ thống hoàn chỉnh có khả năng hoạt động thực tế. Trong đó, cả ba mô hình thành phần đều phải được mô tả, đánh giá và chứng minh vai trò trong chuỗi xử lý tổng thể.

- Tích hợp mô hình nhận diện ký hiệu ASL từ dữ liệu video bằng MediaPipe Holistic và mô hình TFLite pre-trained (Kaggle ASL Signs).
- Xây dựng mô hình hoặc cơ chế khôi phục câu tiếng Việt từ chuỗi gloss đầu ra.
- Xây dựng mô hình dịch Việt - Lào dựa trên kiến trúc dịch máy đã fine-tune.
- Tích hợp ba mô-đun thành hệ thống chạy thử thời gian thực.
- Đánh giá từng mô hình bằng các bộ metric phù hợp và đánh giá toàn hệ thống.

Các mục tiêu trên được triển khai theo hai mức. Ở mức mô-đun, mỗi thành phần cần có đầu vào, đầu ra, cấu trúc mã nguồn và cách đánh giá riêng. Ở mức hệ thống, các mô-đun phải giao tiếp được với nhau thông qua định dạng trung gian rõ ràng, cụ thể là nhân/gloss sau nhận diện và câu tiếng Việt sau khôi phục. Nhờ đó, báo cáo không chỉ trình bày một mô hình đơn lẻ mà thể hiện được toàn bộ quy trình xây dựng ứng dụng AI đa mô hình.

1.4 Phạm vi nghiên cứu

Phạm vi của đề tài được xác định rõ để phù hợp với dữ liệu, tài nguyên tính toán và thời gian thực hiện. Hệ thống trong giai đoạn này tập trung vào bài toán nhận diện theo từ hoặc cụm từ ngắn, sau đó thực hiện khôi phục câu và dịch ở mức câu đơn hoặc câu có cấu trúc tương đối ngắn.

- Mô-đun nhận diện sử dụng mô hình TFLite pre-trained từ cuộc thi Kaggle ASL Signs với 250 lớp từ vựng ASL, chưa phải mô hình VSL thuần được huấn luyện lại từ dữ liệu người Việt.

- Mô-đun khôi phục câu tiếng Việt tập trung vào các chuỗi gloss ngắn, các mẫu giao tiếp cơ bản và các câu có cấu trúc tương đối rõ ràng.
- Mô-đun dịch Việt – Lào sử dụng mô hình NLLB-200 Distilled 600M kết hợp LoRA, đánh giá trên cặp câu song ngữ đã chia sẵn trong repository.
- Hệ thống demo ưu tiên một người dùng, một webcam, điều kiện nền tương đối đơn giản và khoảng cách quay phù hợp để MediaPipe phát hiện được tay.
- Các nội dung như nhận diện hội thoại ký hiệu liên tục dài, nhiều người xuất hiện đồng thời, triển khai mobile, phát giọng nói đầu ra và hội thoại hai chiều chưa thuộc phạm vi chính của phiên bản hiện tại.

1.5 Phương pháp thực hiện

Đề tài được triển khai theo hướng mô-đun hóa. Mỗi mô-đun được xây dựng, kiểm thử và đánh giá độc lập trước khi tích hợp vào pipeline chung. Cách tiếp cận này giúp tách biệt lỗi, thuận tiện tối ưu và làm rõ đóng góp của từng mô hình.

- Thu thập hoặc kế thừa dữ liệu cho từng bài toán con.
- Tiền xử lý dữ liệu và chuẩn hóa đầu vào theo đặc thù mỗi mô hình.
- Huấn luyện, tinh chỉnh, đánh giá riêng từng mô hình.
- Tích hợp ba mô-đun vào chương trình demo thời gian thực.
- So sánh kết quả ở mức mô-đun và mức hệ thống tổng thể.

Về quy trình kỹ thuật, đề tài đi theo thứ tự từ dưới lên. Trước hết, tầng thị giác được kiểm tra độc lập bằng webcam để bảo đảm có thể trích xuất landmarks và sinh nhãn ký hiệu. Tiếp theo, tầng tiếng Việt được kiểm thử bằng chuỗi token nhập tay nhằm loại bỏ ảnh hưởng của lỗi nhận diện. Sau đó, tầng dịch Việt – Lào được kiểm thử bằng câu tiếng Việt hoàn chỉnh. Chỉ sau khi từng tầng hoạt động riêng lẻ, hệ thống mới được ghép thành pipeline tích hợp. Cách làm này giúp xác định rõ lỗi phát sinh ở tầng nào thay vì chỉ quan sát đầu ra cuối cùng.

Trong phần đánh giá, báo cáo kết hợp hai nhóm bằng chứng. Nhóm thứ nhất là bằng chứng định lượng gồm accuracy, loss, BLEU, ROUGE, chrF, TER hoặc các metric tương ứng với từng bài toán. Nhóm thứ hai là bằng chứng định tính gồm quan sát demo, ví dụ đầu vào – đầu ra và phân tích lỗi. Việc kết hợp hai nhóm bằng chứng giúp đánh giá hệ thống thực tế hơn, đặc biệt vì pipeline có nhiều tầng và không phải mô-đun nào cũng có cùng loại dữ liệu kiểm thử.

1.6 Cấu trúc báo cáo

Báo cáo được tổ chức thành bốn chương chính. Chương 1 giới thiệu bối cảnh, phát biểu bài toán, mục tiêu, phạm vi và phương pháp thực hiện. Chương 2 trình bày kiến trúc hệ thống, luồng dữ liệu và kiến trúc chi tiết của ba mô hình cốt lõi: nhận diện ký hiệu, khôi phục câu tiếng Việt và dịch Việt – Lào. Chương 3 mô tả quá trình xây dựng phần mềm, yêu cầu hệ thống, use case, cấu trúc mã nguồn và cách tích hợp các mô-đun. Chương 4 trình bày thiết lập thực nghiệm, các metric đánh giá, phân tích kết quả, kết luận, hạn chế và hướng phát triển tiếp theo của đề tài.

2. KIẾN TRÚC HỆ THỐNG VÀ KIẾN TRÚC CÁC MÔ HÌNH

2.1 Kiến trúc tổng thể hệ thống

Hệ thống được thiết kế theo kiến trúc pipeline nhiều tầng, trong đó đầu ra của mô-đun trước là đầu vào của mô-đun sau. Cấu trúc này phù hợp với bản chất của bài toán vì dữ liệu biến đổi tuần tự từ tín hiệu thị giác sang biểu diễn từ vựng, sau đó sang câu tiếng Việt hoàn chỉnh và cuối cùng sang câu tiếng Lào.

Về tổng thể, hệ thống gồm năm khối chính. Khối thu nhận video đọc khung hình từ webcam và chuyển đổi sang định dạng phù hợp cho MediaPipe. Khối nhận diện ký hiệu trích xuất landmarks, duy trì cửa sổ trượt theo thời gian và dự đoán nhãn ký hiệu bằng mô hình TFLite. Khối khôi phục câu tiếng Việt nhận chuỗi token/gloss, áp dụng rule hoặc mô hình BARTpho để sinh câu tiếng Việt tự nhiên. Khối dịch máy sử dụng NLLB-200 Distilled 600M kết hợp adapter LoRA để dịch câu tiếng Việt sang tiếng Lào. Cuối cùng, khối giao diện hiển thị video, nhãn dự đoán, buffer token, câu tiếng Việt và câu tiếng Lào.

Quan hệ giữa các mô-đun là quan hệ phụ thuộc một chiều. Nếu tầng nhận diện không phát hiện được gloss đúng, tầng NLP sẽ nhận đầu vào sai và có thể sinh câu sai nghĩa. Nếu câu tiếng Việt được khôi phục chưa tốt, mô-đun dịch sẽ tiếp tục khuếch đại lỗi này sang tiếng Lào. Vì vậy, kiến trúc hệ thống không chỉ cần mô hình mạnh ở từng tầng mà còn cần các cơ chế trung gian như ánh xạ nhãn, bộ đệm token, ngưỡng tin cậy và thao tác xác nhận của người dùng.

Hệ thống hỗ trợ hai chế độ vận hành. Chế độ offline/smoke test cho phép nhập token thủ công để kiểm tra riêng tầng NLP và dịch máy. Chế độ realtime dùng webcam để chạy toàn bộ pipeline từ video đến văn bản. Việc tách hai chế độ này giúp kiểm thử thuận tiện hơn: khi một tầng có lỗi, có thể cô lập tầng đó mà không cần chạy lại toàn bộ hệ thống.

2.2 Luồng dữ liệu từ đầu vào đến đầu ra

Luồng dữ liệu của hệ thống được tổ chức tuần tự để bảo đảm mỗi tầng nhận đúng kiểu dữ liệu mà mô hình tương ứng yêu cầu. Mỗi khung hình từ webcam được đưa vào MediaPipe Holistic để trích xuất toàn bộ 543 landmarks, sau đó ghép thành chuỗi thời gian để mô hình TFLite (Kaggle ASL Signs) suy luận nhãn ký hiệu. Chuỗi từ thô tiếp tục được xử lý bởi mô-đun ngôn ngữ để tạo câu tiếng Việt, trước khi chuyển sang mô hình dịch Việt – Lào.

2.2.1 Các bước xử lý chính

- Thu nhận video từ webcam theo thời gian thực.

- Trích xuất keypoints pose, tay trái và tay phải.
- Gom cửa sổ trượt theo số frame cố định.
- Suy luận nhãn ký hiệu và đẩy vào buffer từ vựng.
- Ghép buffer thành câu tiếng Việt.
- Dịch câu tiếng Việt sang tiếng Lào.

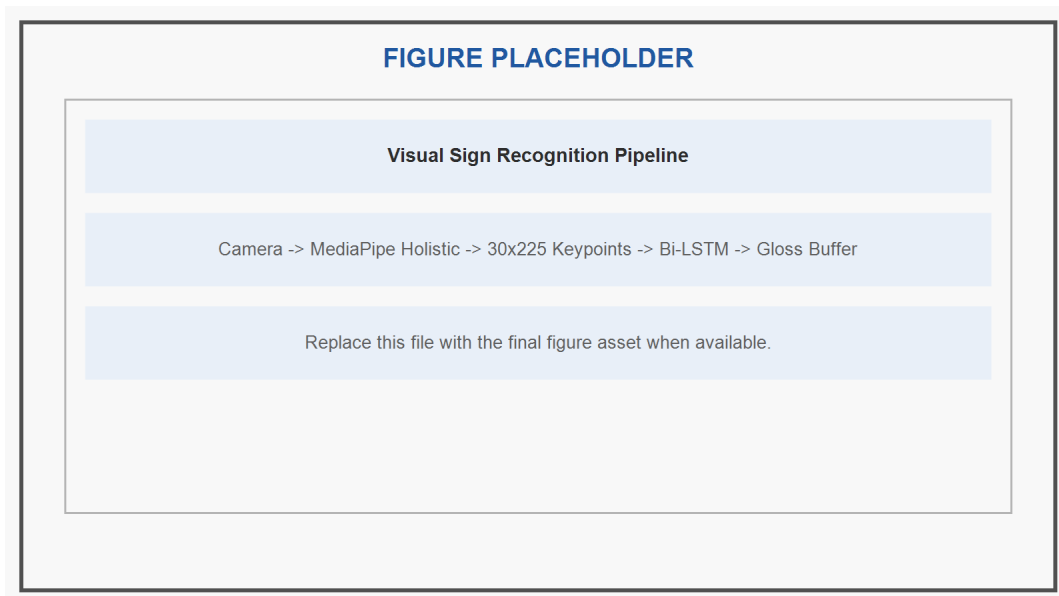
Định dạng dữ liệu thay đổi rõ rệt qua từng bước. Ở đầu vào, dữ liệu là ảnh màu có kích thước phụ thuộc webcam. Sau MediaPipe, mỗi frame trở thành ma trận 543 x 3. Sau tầng nhận diện, dữ liệu được rút gọn thành nhãn ký hiệu như hello hoặc drink. Sau tầng adapter, nhãn này được ánh xạ sang token tiếng Việt như xin_chao hoặc uong_nuoc. Sau tầng NLP, token trở thành câu tiếng Việt hoàn chỉnh. Sau tầng dịch, câu được chuyển sang tiếng Lào.

Việc mô tả rõ các định dạng trung gian là cần thiết vì đây là điểm dễ phát sinh lỗi khi tích hợp. Ví dụ, mô hình nhận diện trả nhãn tiếng Anh theo bộ ASL-250, trong khi mô-đun tiếng Việt yêu cầu token tiếng Việt đã chuẩn hóa. Do đó, hệ thống cần một lớp adapter ánh xạ từ ASL label sang token mà mô-đun NLP hiểu được.

2.3 Mô hình nhận diện ký hiệu ở tầng thị giác

Mô-đun đầu tiên của hệ thống chịu trách nhiệm biến dữ liệu video thành một chuỗi gloss trung gian để đưa sang tầng xử lý ngôn ngữ. Về mặt chức năng, đây là khối “đọc” cử chỉ của người dùng thông qua webcam. Về mặt kỹ thuật, hệ thống sử dụng MediaPipe Holistic để trích xuất toàn bộ landmarks cơ thể từ mỗi frame video, sau đó đưa chuỗi landmarks theo thời gian vào một mô hình TFLite pre-trained (giải pháp đạt giải nhất cuộc thi Kaggle ASL Signs) để dự đoán nhãn ký hiệu.

Cần nhấn mạnh một điểm quan trọng để báo cáo trung thực với hiện trạng dự án: mô-đun này sử dụng mô hình đã được huấn luyện sẵn trên bộ dữ liệu Kaggle ASL Signs gồm 250 từ vựng ký hiệu ASL. Mô hình không được nhóm tự huấn luyện từ đầu, mà được tích hợp như một thành phần pre-trained trong pipeline nhận diện. Sau khi dự đoán nhãn tiếng Anh, hệ thống ánh xạ sang gloss trung gian tiếng Việt để nối với mô-đun NLP phía sau.



Hình 2.1 Vị trí của mô-đun nhận diện thị giác trong pipeline tổng thể

2.3.1 Phát biểu bài toán

Xét một đoạn video ngắn hoặc một cửa sổ trượt gồm nhiều frame liên tiếp từ webcam, mục tiêu của mô hình là dự đoán ra một nhãn từ vựng ký hiệu tương ứng. Trong dự án này, đầu ra của mô hình là một *gloss* ở mức từ hoặc cụm từ ngắn, ví dụ *drink*, *hello*, *help*, *food*. Gloss đó sau đó có thể được ánh xạ sang token tiếng Việt tương ứng.

Do ký hiệu là tín hiệu có tính động, bài toán không thể chỉ nhìn một frame đơn lẻ. Ý nghĩa của một từ ký hiệu thường phụ thuộc đồng thời vào:

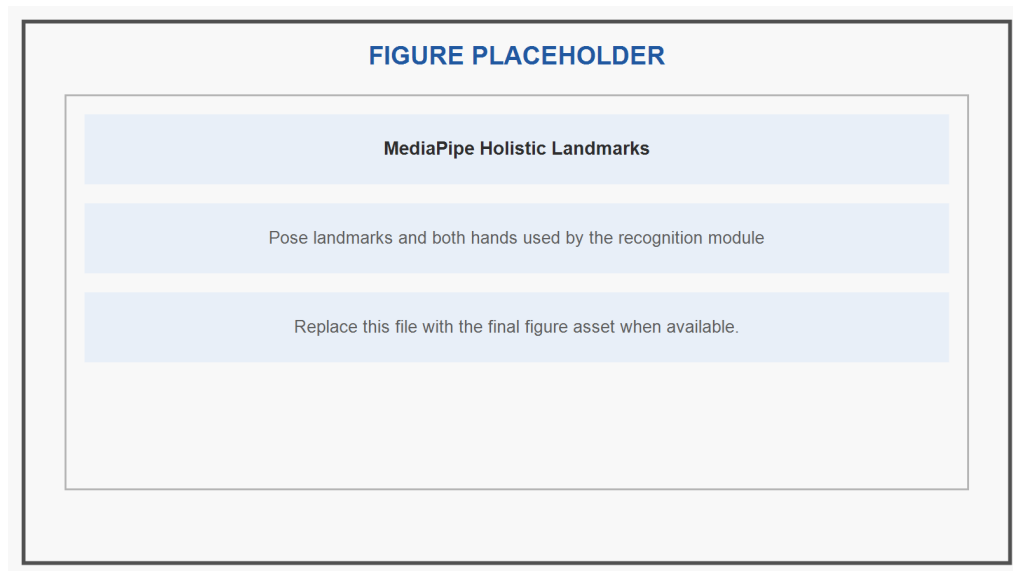
- vị trí tương đối giữa tay, vai, khuỷu tay và thân người;
- hình dạng và quỹ đạo chuyển động của hai bàn tay;
- biểu cảm khuôn mặt và hướng nhìn;
- thứ tự diễn tiến của các động tác theo thời gian.

Vì vậy, mô hình được thiết kế như một bộ phân loại đa lớp trên *chuỗi đặc trưng theo thời gian*, thay vì trên từng ảnh tĩnh.

2.3.2 Vai trò của MediaPipe Holistic trong pipeline

Theo tài liệu chính thức của Google, MediaPipe Holistic có thể xuất tổng cộng 543 landmarks trong thời gian thực, bao gồm 468 điểm khuôn mặt, 33 điểm pose và 21 điểm cho mỗi bàn tay. Khác với phiên bản trước của dự án (chỉ dùng pose và bàn tay với 225 đặc trưng), mô hình hiện tại giữ lại **toàn bộ 543 landmarks** để tận dụng tối đa thông tin về biểu cảm khuôn mặt, thân trên và hai bàn tay. Đây là lựa chọn phù hợp vì:

- mô hình TFLite đã được huấn luyện trên toàn bộ 543 landmarks, nên đầu vào phải giữ nguyên cấu trúc này;
- landmarks khuôn mặt bổ sung thêm tín hiệu biểu cảm quan trọng cho nhiều từ ký hiệu;
- MediaPipe Holistic đã được tối ưu cho luồng ảnh liên tục, phù hợp với bài toán realtime trên webcam.



Hình 2.2 Minh họa landmarks pose, khuôn mặt và bàn tay được trích xuất bởi MediaPipe Holistic

Bảng 2.1 Thành phần vector đặc trưng ở mỗi frame

Thành phần	Số điểm	Số tọa độ/điểm	Tổng đặc trưng	
Khuôn mặt	468	3	1.404	
Tay trái	21	3	63	
Pose	33	3	99	
Tay phải	21	3	63	
Tổng	543	3	1.629	

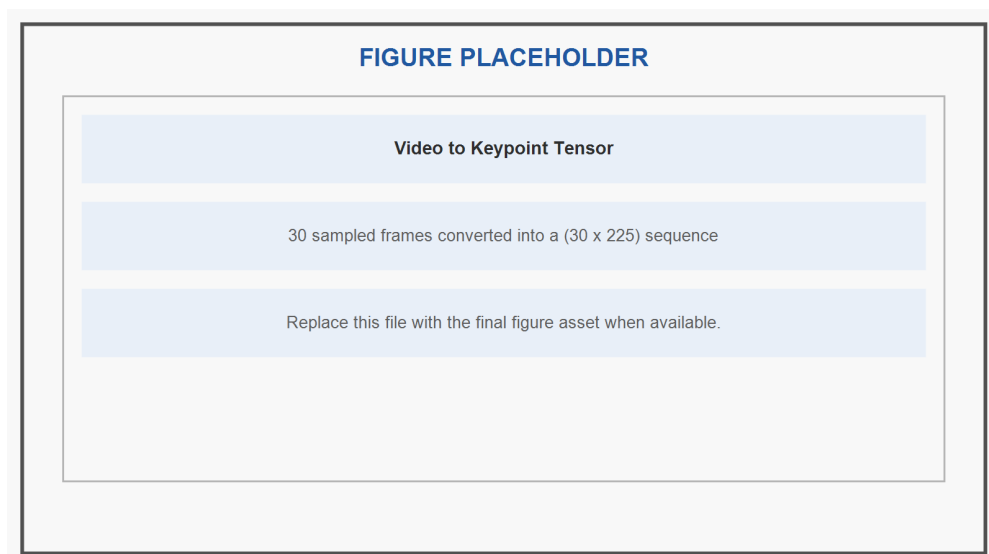
2.3.3 Biểu diễn đầu vào theo chuỗi landmarks

Sau khi một frame được biến thành ma trận landmarks kích thước 543×3 , hệ thống gom các frame liên tiếp thành một chuỗi có độ dài linh hoạt. Trong code hiện tại (`realtime_asl_250.py`), cửa sổ trượt có kích thước tối đa 60 frame và yêu cầu tối thiểu 16 frame trước khi bắt đầu suy luận. Kết quả là mỗi lần suy luận nhận đầu vào là một tensor kích thước $N \times 543 \times 3$, với N nằm trong khoảng $[16, 60]$.

Điểm khác biệt quan trọng so với các thiết kế chuỗi cố định là mô hình TFLite có khả năng nhận đầu vào có chiều dài thay đổi nhờ cơ chế `resize_tensor_input()`. Điều này mang lại hai lợi ích:

- mô hình có thể bắt đầu dự đoán sớm ngay khi có đủ 16 frame, giảm độ trễ ban đầu;
- khi cửa sổ trượt đầy (60 frame), mô hình có nhiều ngữ cảnh hơn để phân loại chính xác.

Một chi tiết kỹ thuật quan trọng trong code là nếu frame bị mất landmark, hệ thống sẽ đệm bằng NaN thay vì vector zero. Đây là quy ước của mô hình Kaggle ASL Signs: giá trị NaN cho phép mô hình phân biệt giữa “không có tay” và “tay ở vị trí gốc tọa độ”.



Hình 2.3 Sơ đồ biến video thành chuỗi landmarks kích thước $N \times 543 \times 3$

2.3.4 Kiến trúc mô hình TFLite (Kaggle ASL Signs 1st Place)

Mô hình nhận diện trong dự án là giải pháp đạt giải nhất cuộc thi *Google – Isolated Sign Language Recognition* trên Kaggle. Mô hình đã được chuyển đổi sang định dạng TensorFlow Lite (`model.tflite`) để tối ưu cho suy luận nhanh trên thiết bị đầu cuối.

Về kiến trúc, mô hình này khác biệt hoàn toàn so với các mạng hồi quy truyền thống:

- đầu vào là chuỗi landmarks có chiều dài linh hoạt, mỗi frame gồm 543 điểm x 3 tọa độ;
- mô hình được huấn luyện end-to-end trên dữ liệu landmarks, không cần bước trích xuất đặc trưng thủ công;

- đầu ra là phân bố xác suất trên 250 lớp từ vựng ký hiệu ASL;
- mô hình sử dụng TFLite Interpreter cho suy luận, cho phép chạy trên CPU mà không cần GPU.

Trong code, quá trình suy luận được thực hiện qua các bước:

1. gom landmarks thành tensor đầu vào;
2. gọi `interpreter.resize_tensor_input()` để điều chỉnh kích thước đầu vào theo số frame thực tế;
3. gọi `interpreter.allocate_tensors()` và `set_tensor()`;
4. gọi `interpreter.invoke()` để chạy suy luận;
5. lấy kết quả từ `get_tensor()`, áp dụng softmax nếu cần;
6. chọn top-k lớp có xác suất cao nhất.

File `sign_to_prediction_index_map.json` chứa bảng ánh xạ giữa tên từ vựng ký hiệu và chỉ số đầu ra của mô hình, cho phép chuyển đổi kết quả số thành nhãn có nghĩa.

Bảng 2.2 Thông số chính của mô hình nhận diện

Tham số	Giá trị trong code
Số lớp từ vựng	250
Kích thước đầu vào mỗi frame	543 landmarks x 3 tọa độ
Số frame tối thiểu	16
Số frame tối đa (sliding window)	60
Tần suất suy luận	Mỗi 8 frame
Top-k hiển thị	3
Định dạng mô hình	TensorFlow Lite (.tflite)
Kích thước file mô hình	~11 MB

2.3.5 Nguồn gốc mô hình và dữ liệu huấn luyện

Mô hình TFLite được lấy từ giải pháp đạt giải nhất cuộc thi Kaggle *Google – Isolated Sign Language Recognition*. Cuộc thi này yêu cầu các đội thi xây dựng mô

hình nhận diện 250 từ vựng ký hiệu ASL từ dữ liệu landmarks đã được trích xuất bằng MediaPipe Holistic.

Dữ liệu huấn luyện gốc của cuộc thi bao gồm:

- 250 từ vựng ký hiệu ASL phổ biến;
- dữ liệu landmarks đã được trích xuất sẵn (không phải video RGB);
- nhiều signer với điều kiện ghi hình khác nhau.

Vì đây là mô hình pre-trained, nhóm không thực hiện quá trình huấn luyện lại. Thay vào đó, mô hình được sử dụng trực tiếp làm thành phần nhận diện trong pipeline tổng thể. Đây là một lựa chọn thực dụng cho đề án vì cho phép tập trung nguồn lực vào các tầng NLP phía sau, đồng thời vẫn đảm bảo chất lượng nhận diện ở mức cạnh tranh nhờ vào kết quả đã được kiểm chứng qua cuộc thi.

2.3.6 Cơ chế suy luận thời gian thực

Trong `realtime_asl_250.py`, hệ thống duy trì một bộ đệm trượt:

- `frames = deque(maxlen=60);`
- chỉ gom frame khi phát hiện có bàn tay (trái hoặc phải) xuất hiện trong khung hình;
- suy luận mỗi 8 frame (`predict_every=8`);
- chỉ bắt đầu suy luận khi bộ đệm đủ tối thiểu 16 frame;
- hiển thị top-3 dự đoán có xác suất cao nhất;
- nhấn phím Space để reset bộ đệm, phím q để thoát.

Thiết kế “chỉ gom khi có tay” là một cải tiến quan trọng: thay vì gom mọi frame kể cả khi người dùng không ký hiệu, hệ thống chỉ thu thập dữ liệu khi phát hiện ít nhất một bàn tay. Điều này giúp giảm nhiễu đầu vào và tăng chất lượng dự đoán.

2.3.7 Ưu điểm và giới hạn của thiết kế hiện tại

Thiết kế *MediaPipe Holistic + TFLite pre-trained* có một số ưu điểm rõ ràng:

- mô hình đã được kiểm chứng qua cuộc thi Kaggle với chất lượng cao;
- sử dụng toàn bộ 543 landmarks bao gồm cả biểu cảm khuôn mặt;
- kiến trúc gọn, suy luận bằng TFLite trên CPU, phù hợp với phần cứng phổ thông;

- cửa sổ trượt linh hoạt 16–60 frame và logic “chỉ gom khi có tay” phù hợp cho demo realtime;
- đầu ra gloss là giao diện trung gian thuận lợi để nối với tầng NLP.

Mặt khác, vẫn còn các giới hạn quan trọng:

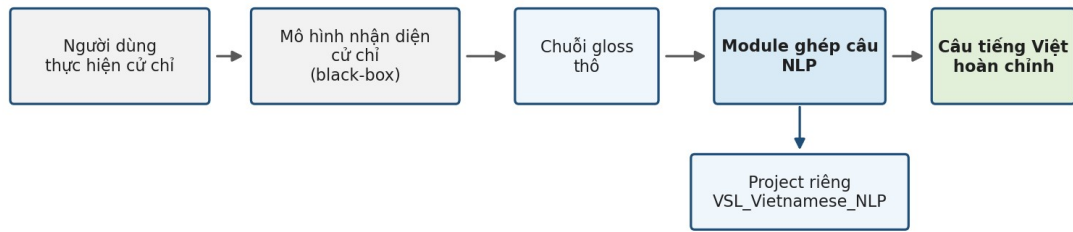
- mô hình được huấn luyện trên dữ liệu ASL, không phải VSL gốc, nên tính đúng đắn ngôn ngữ còn phụ thuộc bước ánh xạ gloss;
- số lớp từ vựng là 250, ít hơn so với nhu cầu giao tiếp thực tế;
- mô hình hiện nhận diện ở mức từ đơn lẻ, chưa xử lý trực tiếp câu ký hiệu liên tục;
- vì là mô hình pre-trained, nhóm không thể can thiệp vào kiến trúc bên trong hoặc tinh chỉnh cho VSL;
- chất lượng cuối cùng của pipeline chịu ảnh hưởng mạnh từ ánh sáng, góc quay, che khuất tay và sự khác biệt giữa signer thật với signer trong dữ liệu Kaggle.

2.4 Mô hình ghép từ thành câu tiếng Việt

Sau khi tầng nhận diện ký hiệu dự đoán ra các gloss hoặc cụm gloss theo thời gian, hệ thống vẫn chưa thể giao tiếp trực tiếp với người dùng cuối. Chuỗi đầu ra ở giai đoạn này thường chỉ là các đơn vị ý nghĩa rời rạc như *toi muon uong_nuoc*, *toi can giup_do* hoặc *toi dau bung*. Vì vậy, cần có một mô-đun xử lý ngôn ngữ tự nhiên để tái cấu trúc chuỗi đó thành câu tiếng Việt hoàn chỉnh, có chủ ngữ, vị ngữ, dấu câu và cách diễn đạt tự nhiên hơn.

Về bản chất, đây là bài toán *sequence-to-sequence*: đầu vào là chuỗi gloss đã được chuẩn hóa, đầu ra là câu tiếng Việt hoàn chỉnh. Trong các nghiên cứu gần đây về dịch ngôn ngữ ký hiệu, pha *Gloss-to-Text* vẫn được xem là một bước quan trọng để nối tầng thị giác với tầng sinh ngôn ngữ, và thường được đánh giá bằng các metric như BLEU, ROUGE hoặc chrF. Trong đồ án này, thay vì dùng trực tiếp một mô hình đa ngôn ngữ, nhóm lựa chọn một mô hình sinh văn bản đơn ngữ dành riêng cho tiếng Việt để tối ưu chất lượng diễn đạt ở ngôn ngữ đích.

Vị trí module ghép câu trong hệ thống



Phạm vi chương: xử lý gloss và sinh câu tiếng Việt; phần nhận diện cử chỉ chỉ được xem như nguồn cung cấp gloss.

Hình 2.4 Vị trí của mô-đun ghép câu trong hệ thống xử lý ký hiệu

2.4.1 Phát biểu bài toán

Cho một chuỗi gloss đầu vào nhận được từ tầng nhận diện ký hiệu, mô-đun ghép câu cần sinh ra một chuỗi đầu ra là câu tiếng Việt hoàn chỉnh sao cho:

- bảo toàn ý nghĩa cốt lõi của chuỗi gloss đầu vào;
- bổ sung được các thành phần ngữ pháp cần thiết cho tiếng Việt;
- giảm tính rời rạc, lặp từ và thiếu dấu câu của đầu ra trung gian;
- đủ nhẹ để tích hợp vào chế độ realtime.

Ví dụ, chuỗi *toi muon uong_nuoc* không nên chỉ được hiển thị nguyên dạng, mà cần được chuyển thành câu *Tôi muốn uống nước*. Tương tự, *toi can giup_do* nên được chuyển thành *Tôi cần giúp đỡ*.

2.4.2 Kiến trúc lai được sử dụng

Thay vì phụ thuộc hoàn toàn vào một mô hình sinh văn bản, project hiện thực mô-đun này theo kiến trúc lai gồm ba tầng ưu tiên:

1. **Rule-based/template**: tra cứu các mẫu câu có tần suất cao trong `configs/sentence_rules.json`. Đây là tầng có độ ổn định cao nhất và phản hồi rất nhanh.
2. **BARTpho fine-tuned**: nếu chuỗi đầu vào không khớp rule cứng, hệ thống dùng mô hình sinh văn bản để phục hồi câu tiếng Việt linh hoạt hơn.
3. **Fallback corrector**: nếu mô hình chưa được huấn luyện hoặc không thể nạp, hệ thống vẫn còn tầng dự phòng để ánh xạ token cơ bản và thêm dấu câu tối thiểu.

Thiết kế này bám sát hiện thực trong mã nguồn: lớp SentenceBuilder lần lượt kiểm tra rule, gọi predictor BARTpho và cuối cùng rơi về tầng fallback. Cách tổ chức như vậy giúp mô-đun vừa có khả năng sinh câu tự nhiên, vừa đảm bảo tính an toàn khi triển khai thực tế.

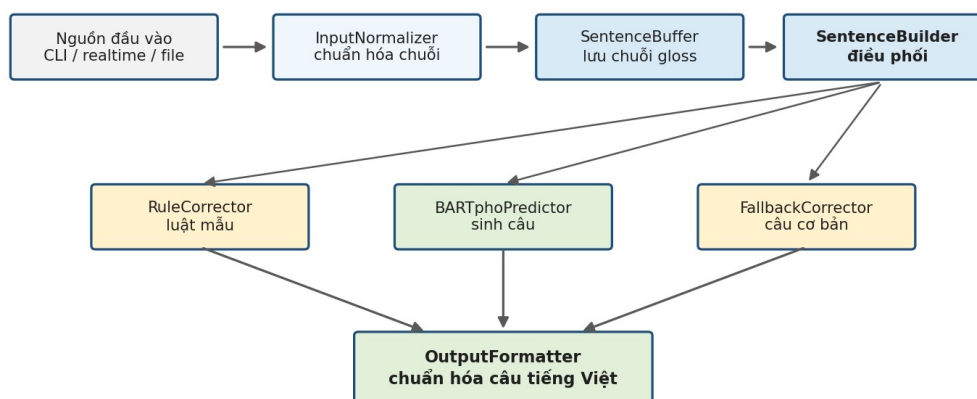
Pipeline xử lý từ gloss thô đến câu tiếng Việt



Pipeline tách biệt với mô hình nhận diện cử chỉ; module chỉ cần nhận chuỗi gloss đã ánh xạ.

Hình 2.5 Pipeline xử lý từ gloss thô đến câu tiếng Việt hoàn chỉnh

Kiến trúc các thành phần của module ghép câu



Hình 2.6 Kiến trúc các thành phần chính của mô-đun ghép câu

Bảng 2.3 Vai trò của các thành phần trong mô-đun ghép câu

Thành phần	Vai trò chính	Ý nghĩa trong hệ thống realtime
Rule/template	Xử lý các mẫu giao tiếp ngắn, lặp lại nhiều lần như chào hỏi, xin lỗi, yêu cầu hỗ trợ.	Phản hồi gần như tức thời, giảm phụ thuộc vào mô hình sinh.
BARTpho fine-tuned	Khôi phục câu tự nhiên cho các chuỗi gloss linh hoạt hoặc chưa có rule.	Tăng độ tự nhiên và khả năng tổng quát hóa của đầu ra.
Fallback corrector	Ánh xạ token cơ bản, viết hoa đầu câu và thêm dấu câu tối thiểu.	Đảm bảo hệ thống vẫn sử dụng được khi mô hình chưa sẵn sàng.

2.4.3 Lý do chọn BARTpho

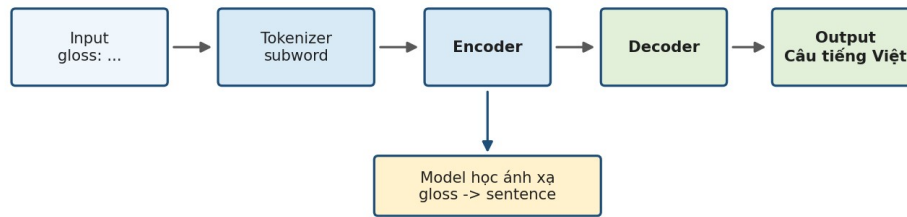
BARTpho là mô hình *sequence-to-sequence* đơn ngữ đầu tiên được tiền huấn luyện quy mô lớn cho tiếng Việt. Theo Tran, Le và Nguyen, BARTpho có hai biến thể là *BARTpho-syllable* và *BARTpho-word*; mô hình sử dụng kiến trúc *BART-large* và đặc biệt phù hợp với các tác vụ sinh văn bản tiếng Việt. Về gốc, BART là một *denoising autoencoder* kết hợp encoder hai chiều và decoder sinh trái sang phải, nhờ đó rất phù hợp cho các tác vụ phục hồi câu, tóm tắt hoặc tái cấu trúc chuỗi.

Trong project này, cấu hình `configs/nlp_config.json` chọn mô hình nền BARTpho word-base. Đây là lựa chọn hợp lý vì:

- đầu vào của mô-đun đã ở dạng token từ hoặc cụm từ phân cách bằng khoảng trắng;
- nhiều gloss dùng dấu gạch dưới để biểu diễn cụm từ cố định như `uong_nuoc`, `benh_vien`, `giup_do`;
- mô hình đơn ngữ tiếng Việt thường diễn đạt câu đích mượt hơn so với mô hình đa ngôn ngữ tổng quát khi tác vụ chỉ có một ngôn ngữ đầu ra.

Để đồng nhất giữa huấn luyện và suy luận, project thêm tiền tố nhiệm vụ `chuyen_cau_ky_hieu`: vào chuỗi đầu vào trước khi token hóa. Đây là một chi tiết quan trọng vì giúp mô hình hiểu đúng ngữ cảnh nhiệm vụ sinh câu, đồng thời tránh lệch phân phối giữa train và inference.

Kiến trúc mô hình sinh câu sử dụng BARTpho



BARTpho là mô hình xử lý ngôn ngữ, không nhận diện hình ảnh/cử chỉ.

Hình 2.7 Kiến trúc sinh câu tiếng Việt bằng BARTpho

2.4.4 Thông số thiết kế chính

Các thông số nổi bật đang được dùng trong project gồm:

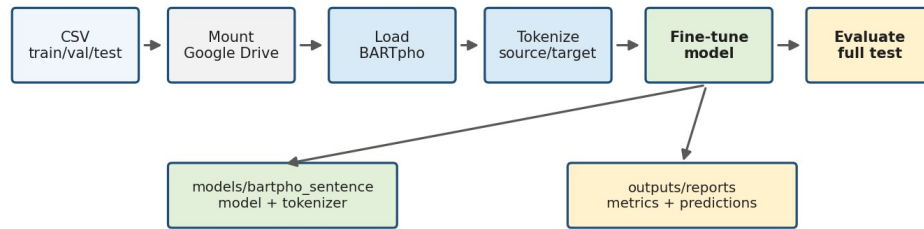
- mô hình nền: `vinai/bartpho-word-base`;
- độ dài tối đa đầu vào: 64 token;
- độ dài tối đa đầu ra: 64 token;
- huấn luyện với AdamW, learning rate $5e-5$;
- suy luận dùng beam search với `num_beams = 4`.

Bên cạnh cấu hình suy luận/demo đang lưu trong repository, tài liệu `vs1.pdf` ghi nhận lần fine-tune chính của BARTpho được thực hiện trên Google Colab với cấu hình dài hơn cho chuỗi nguồn và chuỗi đích. Cấu hình này được dùng làm căn cứ cho phần đánh giá ở Chương 4.

Bảng 2.4 Cấu hình fine-tune BARTpho

Thành phần	Giá trị
Môi trường	Google Colab
GPU	T4
Mô hình gốc	BARTpho
Số epoch	5
Batch size	4
Max source length	96
Max target length	128

Quy trình huấn luyện và đánh giá BARTpho trên Colab



Hình 2.8 Quy trình huấn luyện và đánh giá BARTpho

Tập luật hiện tại đóng vai trò như một lớp tri thức miền hẹp, đặc biệt hữu ích với các câu giao tiếp chăm sóc sức khỏe hoặc nhu cầu cơ bản. Khi gloss đầu vào chưa có trong tập luật, BARTpho đảm nhiệm phần tổng quát hóa. Nếu cả hai đường này đều không khả dụng, fallback sẽ duy trì đầu ra đọc được thay vì làm hỏng toàn bộ pipeline.

2.4.5 Ưu điểm và giới hạn

Ưu điểm lớn nhất của kiến trúc hiện tại là cân bằng giữa *độ ổn định* và *độ tự nhiên*. Rule-based giúp giữ đúng các mẫu câu quan trọng, còn BARTpho cho phép mô-đun không bị giới hạn ở một tập câu cứng. Điều này đặc biệt phù hợp với hệ thống thời gian thực, nơi lỗi của tầng nhận diện trước đó có thể lan sang tầng NLP.

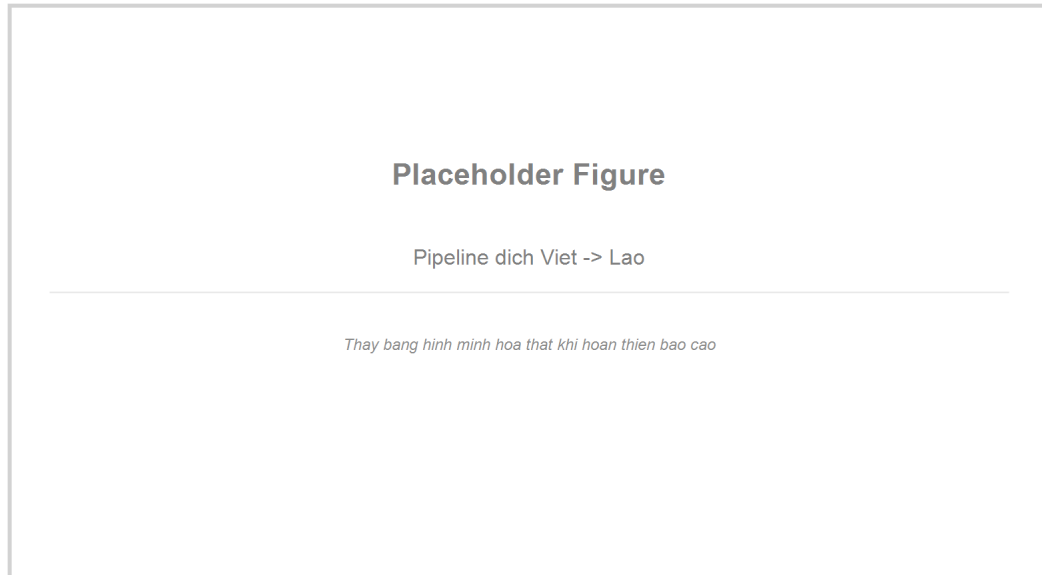
Tuy nhiên, mô-đun này vẫn có một số giới hạn:

- nếu chuỗi gloss đầu vào sai quá nhiều do nhận diện nhầm, mô hình sinh câu vẫn có thể bảo toàn lỗi ngữ nghĩa;
- tập luật hiện tại phù hợp nhất với miền giao tiếp cơ bản, chưa đủ bao phủ hội thoại mở;
- chất lượng metric rất cao trên bộ dữ liệu sinh theo template không đồng nghĩa với khả năng tổng quát hóa hoàn toàn ngoài miền dữ liệu đó.

2.5 Mô hình dịch Việt - Lào

Tầng thứ ba của hệ thống nhận câu tiếng Việt đã được chuẩn hóa từ mô-đun trước và dịch sang tiếng Lào. Nếu mô-đun ghép câu giúp đầu ra trở nên tự nhiên trong nội ngữ, thì mô-đun này mở rộng hệ thống sang giao tiếp liên ngôn ngữ. Trong bối cảnh của đề án, đây là khâu biến một pipeline nhận diện ký hiệu thành một pipeline truyền đạt thông tin cho người dùng ở ngôn ngữ khác.

Tài liệu mô tả ban đầu trong `vilo.pdf` trình bày các khái niệm nền như *Sequence-to-Sequence*, *Attention*, *Transformer* và *Embedding*. Đối chiếu với mã nguồn thực tế, project hiện tại hiện thực hóa các ý tưởng đó bằng mô hình NLLB-200 Distilled 600M kết hợp adapter LoRA, thay vì dùng một kiến trúc Seq2Seq tự xây từ đầu.



Hình 2.9 Vị trí của mô-đun dịch Việt - Lào trong pipeline tổng thể

2.5.1 Bài toán dịch máy trong phạm vi đề tài

Đầu vào của mô-đun là câu tiếng Việt đã qua tầng sửa ngữ pháp, ví dụ *Tôi muốn uống nước.* hoặc *Tôi cần giúp đỡ.* Đầu ra mong muốn là câu tiếng Lào tương đương về nghĩa, đủ tự nhiên để có thể hiển thị hoặc kết hợp với các tầng hỗ trợ giao tiếp khác như text-to-speech.

Xét về mặt học máy, đây là bài toán dịch máy *text-to-text*. Mô hình không chỉ cần dịch đúng từ vựng, mà còn phải bảo toàn vai nghĩa, trật tự thông tin và cách dùng từ theo đúng ngữ cảnh. Điều này đặc biệt quan trọng khi hệ thống phục vụ các tình huống giao tiếp hỗ trợ, nơi một sai lệch nhỏ về nghĩa có thể làm thay đổi ý định của người dùng.

2.5.2 Lý do chọn NLLB-200 Distilled 600M

NLLB (*No Language Left Behind*) là họ mô hình dịch máy đa ngôn ngữ do Meta phát triển với mục tiêu hỗ trợ các ngôn ngữ tài nguyên thấp. Theo NLLB Team, công trình này tập trung thu hẹp khoảng cách hiệu năng giữa ngôn ngữ giàu và nghèo tài nguyên, đồng thời đánh giá trên hơn 40.000 hướng dịch và hướng tới mốc 200 ngôn ngữ. Với một đề tài có cặp ngôn ngữ Việt - Lào, đây là một lựa chọn hợp lý vì tiếng Lào là ngôn ngữ ít tài nguyên hơn nhiều so với các cặp dịch phổ biến như Anh - Pháp hoặc Anh - Đức.

Project sử dụng biến thể `nllb-200-distilled-600M`. So với các bản lớn hơn, bản

distilled 600M cho phép:

- giảm yêu cầu phần cứng, phù hợp hơn với môi trường triển khai học thuật;
- giữ được lợi thế của tiền huấn luyện đa ngôn ngữ;
- thuận tiện hơn khi tinh chỉnh bằng adapter thay vì full fine-tuning.

Vì đầu vào của mô-đun đã là câu tiếng Việt sạch ngữ pháp, NLLB đóng vai trò như một khối dịch máy chuyên dụng nằm ở cuối pipeline, tận dụng kiến thức đa ngôn ngữ có sẵn thay vì phải xây toàn bộ mô hình dịch mới từ đầu.

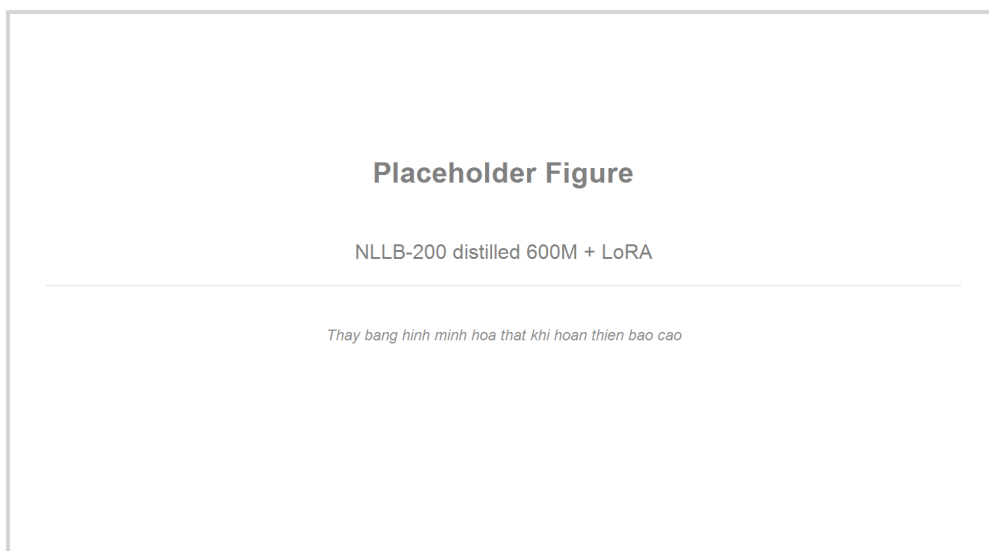
2.5.3 Chiến lược tinh chỉnh bằng LoRA

LoRA (*Low-Rank Adaptation*) là kỹ thuật tinh chỉnh tham số hiệu quả, trong đó mô hình gốc được giữ cố định và chỉ thêm các ma trận hạng thấp có khả năng học vào một số lớp của Transformer. Theo Hu và cộng sự, cách làm này giúp giảm mạnh số tham số cần cập nhật nhưng vẫn duy trì chất lượng tốt trên các tác vụ downstream.

Tệp `adapter_config.json` trong project cho thấy adapter hiện tại có cấu hình:

- `r = 16`;
- `lora_alpha = 32`;
- `lora_dropout = 0.05`;
- áp dụng trên các module `q_proj`, `k_proj`, `v_proj` và `out_proj`;
- kiểu tác vụ: `SEQ_2_SEQ_LM`.

Như vậy, project không huấn luyện lại toàn bộ NLLB mà chỉ học một lớp thích nghi mỏng trên đỉnh mô hình nền. Đây là lựa chọn rất phù hợp với đồ án sinh viên vì vừa tiết kiệm tài nguyên, vừa cho phép lưu trữ mô hình dưới dạng adapter nhỏ gọn.



Hình 2.10 Kiến trúc NLLB-200 kết hợp adapter LoRA trong mô-đun dịch Việt - Lào

2.5.4 Tokenizer và cơ chế sinh câu đích

Theo tài liệu chính thức của Hugging Face cho NLLB, tokenizer của họ mô hình này sử dụng mã ngôn ngữ nguồn và ngôn ngữ đích như các *special tokens*. Trong project, quá trình suy luận được cấu hình như sau:

- ngôn ngữ nguồn: `vie_Latn`;
- ngôn ngữ đích: `lao_Lao`;
- ép token bắt đầu của câu đích bằng `forced_bos_token_id`;
- `max_length = 128`;
- `num_beams = 5`.

Thiết kế này rất quan trọng. Nếu không đặt đúng mã ngôn ngữ hoặc không ép token bắt đầu của tiếng đích, mô hình đa ngôn ngữ có thể sinh sai ngôn ngữ hoặc lẫn giữa nhiều miền dữ liệu. Việc dùng beam search với 5 nhánh cũng cho phép mô hình ưu tiên những câu dịch ổn định hơn so với giải mã tham lam đơn giản.

2.5.5 Ưu điểm và những rủi ro cần lưu ý

Điểm mạnh của hướng tiếp cận hiện tại là tận dụng được tri thức tiền huấn luyện đa ngôn ngữ của NLLB, đồng thời dùng LoRA để hạ chi phí huấn luyện và lưu trữ. Điều này khiến module dịch phù hợp với bối cảnh nghiên cứu ứng dụng, nơi khối lượng phần cứng thường không đủ để full fine-tune các mô hình dịch cỡ lớn.

Tuy nhiên, mô-đun vẫn tồn tại một số rủi ro kỹ thuật:

- chất lượng dịch phụ thuộc trực tiếp vào câu tiếng Việt đầu vào; nếu mô-đun ghép câu phía trước diễn đạt sai ý, mô-đun dịch sẽ khuếch đại lỗi đó;
- dữ liệu song ngữ hiện có khá dị miên, bao gồm hội thoại, câu kể, tên riêng và các câu mô tả dài, nên dễ phát sinh lỗi với thực thể riêng hoặc câu khẩu ngữ;
- một số mẫu dữ liệu quan sát được cho thấy còn hiện tượng nhiều chính tả, viết hoa hoặc phiên âm tên riêng, có thể làm giảm tính ổn định của đầu ra.

Vì vậy, khi đánh giá mô-đun này, không nên chỉ nhìn vào một ví dụ dịch thành công, mà cần đánh giá đồng thời bằng các metric dịch máy chuẩn và các ví dụ định tính nhiều miên.

2.6 Cơ chế tích hợp giữa ba mô hình

Việc tích hợp ba mô hình là nội dung quan trọng của đề tài vì chất lượng hệ thống không chỉ phụ thuộc vào từng mô-đun riêng lẻ mà còn phụ thuộc vào cách chúng truyền dữ liệu cho nhau. Trong project hiện tại, điểm ghép nối giữa các mô-đun được thiết kế theo kiểu dữ liệu trung gian đơn giản, dễ kiểm tra bằng mắt và dễ ghi log.

Tầng nhận diện trả về danh sách top-k nhãn ký hiệu kèm xác suất. Người dùng hoặc chương trình chọn nhãn phù hợp để đưa vào bộ đệm token. Vì mô hình nhận diện hiện tại là ASL-250, hệ thống cần bảng ánh xạ ASL_TO_VI_TOKEN để chuyển nhãn tiếng Anh sang token tiếng Việt nội bộ. Ví dụ, nhãn hello được ánh xạ thành xin_chao; nhãn drink có thể được ánh xạ thành token liên quan đến nhu cầu uống nước. Lớp ánh xạ này đóng vai trò adapter giữa miền thị giác ASL và miền NLP tiếng Việt.

Tầng khôi phục câu nhận bộ đệm token đã chuẩn hóa. Khi người dùng xác nhận, bộ đệm được nối thành chuỗi đầu vào cho SentenceBuilder. Kết quả sinh ra là một câu tiếng Việt đã viết hoa và có dấu câu. Câu này tiếp tục được chuyển sang mô-đun dịch Việt – Lào. Nếu tắt tùy chọn dịch để chạy nhẹ hơn, pipeline vẫn hiển thị câu tiếng Việt và bỏ qua tầng NLLB.

Để giảm lỗi dây chuyền, hệ thống áp dụng một số chiến lược thực dụng: chỉ gom frame khi MediaPipe phát hiện tay, chỉ dự đoán khi đủ số frame tối thiểu, hiển thị top-k thay vì chỉ một nhãn, cho phép người dùng xác nhận token trước khi ghép câu và có chế độ chạy không tải mô hình dịch. Các cơ chế này không loại bỏ hoàn toàn sai số nhưng giúp quá trình demo ổn định hơn, đồng thời giúp xác định rõ lỗi xuất hiện ở tầng nhận diện, tầng ghép câu hay tầng dịch.

2.7 Dữ liệu huấn luyện và tiền xử lý

Ba mô-đun trong hệ thống sử dụng ba kiểu dữ liệu khác nhau: video RGB cho tầng nhận diện, chuỗi gloss – câu tiếng Việt cho tầng ghép câu, và cặp câu song ngữ Việt – Lào cho tầng dịch máy. Vì vậy, phần này không thể chỉ liệt kê tên dataset, mà cần mô tả rõ nguồn dữ liệu, cách tổ chức mẫu, cách chia tập và các bước tiền xử lý cho từng mô-đun. Đây là cơ sở để người đọc đánh giá tính hợp lệ của quá trình huấn luyện và thực nghiệm.

2.7.1 Mô hình pre-trained cho mô-đun nhận diện ký hiệu

Khác với hai mô-đun NLP phía sau (có quá trình huấn luyện rõ ràng trên dữ liệu riêng), mô-đun nhận diện ký hiệu sử dụng một mô hình đã được huấn luyện sẵn (pre-trained). Cụ thể, hệ thống tích hợp mô hình TFLite từ giải pháp đạt giải nhất cuộc thi Kaggle Google – *Isolated Sign Language Recognition*.

Dữ liệu huấn luyện gốc của mô hình bao gồm:

- 250 từ vựng ký hiệu ASL phổ biến;
- dữ liệu landmarks đã được trích xuất bằng MediaPipe Holistic (543 landmarks $\times$ 3 tọa độ);
- nhiều signer với điều kiện ghi hình đa dạng.

Bảng 2.5 Thông tin mô hình pre-trained cho mô-đun nhận diện ký hiệu

Thuộc tính	Giá trị
Nguồn mô hình	Kaggle ASL Signs 1st Place Solution
Số lớp từ vựng	250
Định dạng mô hình	TensorFlow Lite (.tflite)
Kích thước file	~11 MB
Đầu vào mỗi frame	543 landmarks $\times$ 3 tọa độ
Sliding window	16–60 frames

Vì nhóm không tự huấn luyện mô hình này, phần dữ liệu cho mô-đun nhận diện không có train/validation/test split do nhóm thực hiện. Thay vào đó, các file cần thiết để chạy mô-đun nhận diện bao gồm:

- `model.tflite`: trọng số mô hình đã huấn luyện;
- `sign_to_prediction_index_map.json`: bảng ánh xạ giữa tên từ vựng ký hiệu và chỉ số đầu ra;

- `realtime_asl_250.py`: script suy luận thời gian thực.

Đây là một lựa chọn thiết kế có chủ đích: thay vì đầu tư toàn bộ thời gian vào việc tự huấn luyện một mô hình nhận diện từ đầu (với rủi ro chất lượng không đảm bảo), nhóm lựa chọn tích hợp một mô hình đã được kiểm chứng qua cuộc thi Kaggle, từ đó tập trung nguồn lực cho các tầng NLP phía sau.

2.7.2 Bộ dữ liệu cho mô-đun ghép câu tiếng Việt

Theo tài liệu kỹ thuật trong `vs1.pdf`, mô-đun ghép câu được huấn luyện trên bộ dữ liệu *VSL Vietnamese NLP Dataset v4 Mega*. Bộ dữ liệu này bao gồm các cặp `raw_sentence` và `complete_sentence`, trong đó:

- `raw_sentence` là chuỗi gloss hoặc chuỗi từ thô được chuẩn hóa từ ngôn ngữ ký hiệu;
- `complete_sentence` là câu tiếng Việt hoàn chỉnh dùng làm nhãn đích.

Cấu trúc bộ dữ liệu gloss-to-sentence



Hai cột chính dùng cho huấn luyện; metadata dùng để thống kê, phân tích và chia tập dữ liệu.

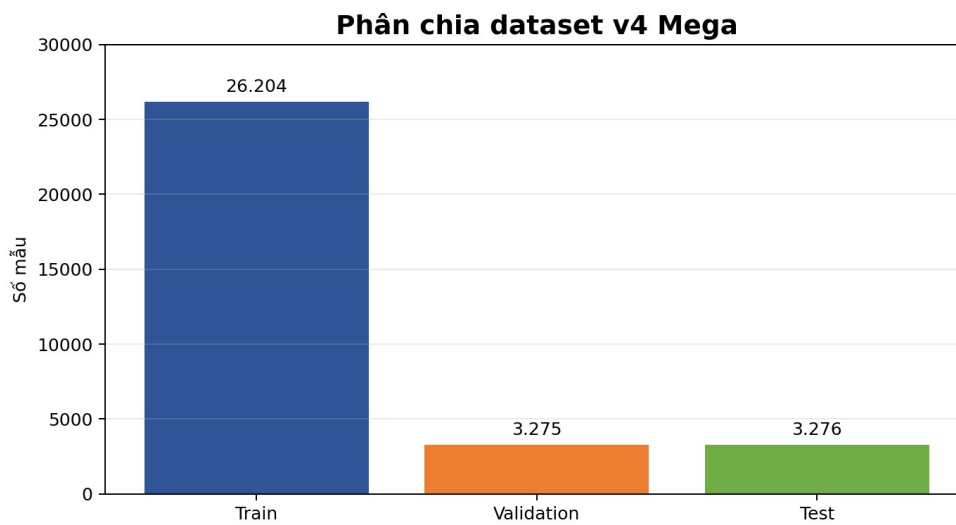
Hình 2.11 Cấu trúc một mẫu dữ liệu gloss-to-sentence

Bảng 2.6 Thống kê bộ dữ liệu VSL Vietnamese NLP Dataset v4 Mega

Thuộc tính	Giá trị
Tổng số mẫu câu	32.755
Số gloss token	262
Số nhóm ngữ cảnh	32
Train set	26.204
Validation set	3.275
Test set	3.276

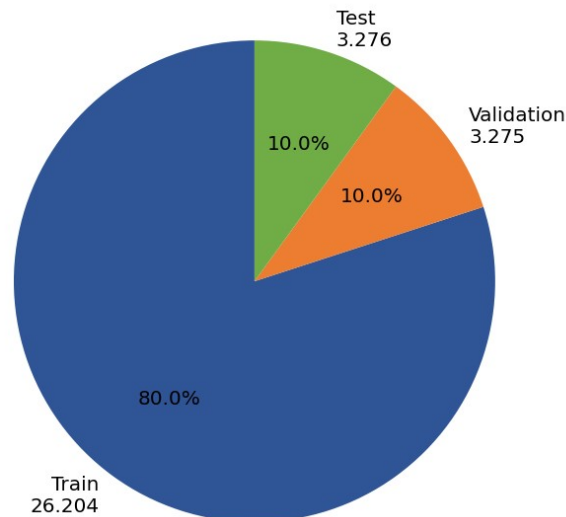
Bên cạnh bộ dữ liệu đầy đủ được mô tả trong tài liệu, snapshot mã nguồn hiện tại chỉ giữ lại một tệp `data/sentence_pairs.csv` gồm 50 dòng mẫu để phục vụ việc kiểm

thử tích hợp và minh họa cục bộ. Điều này cho thấy repository đang lưu phần tối thiểu cần cho demo, còn bộ dữ liệu đầy đủ được quản lý bên ngoài ở giai đoạn huấn luyện chính thức.



Hình 2.12 Số lượng mẫu trong các tập train, validation và test

Tỷ lệ train/validation/test



Hình 2.13 Tỷ lệ chia dữ liệu train, validation và test

2.7.3 Tiền xử lý cho mô-đun ghép câu

Tiền xử lý của mô-đun ghép câu được thực hiện theo đúng bản chất của dữ liệu gloss:

- loại bỏ dòng rỗng hoặc giá trị NaN;
- chuẩn hóa khoảng trắng đầu cuối và khoảng trắng thừa giữa các token;
- giữ lại các cụm từ ghép bằng dấu gạch dưới như `uong_nuoc`, `giup_do`, `benh_vien` để không làm mất đơn vị ý nghĩa;
- thêm tiền tố tác vụ `chuyen_cau_ky_hieu`: vào đầu vào khi train và inference;
- token hóa bằng tokenizer của `vinai/bartpho-word-base`;
- giới hạn độ dài đầu vào và đầu ra ở mức 64 token.

Việc giữ nguyên cấu trúc token dạng gloss là rất quan trọng. Nếu tách sai các cụm `uong_nuoc` hoặc `nha_ve_sinh`, mô hình sẽ phải học lại quan hệ ngữ nghĩa phức tạp hơn và giảm tính ổn định trong quá trình sinh câu.

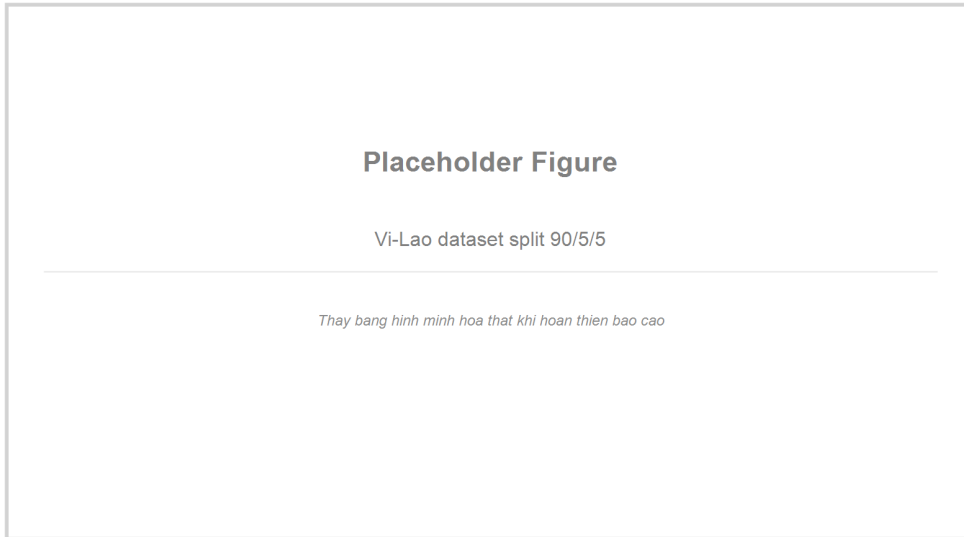
2.7.4 Bộ dữ liệu cho mô-đun dịch Việt - Lào

Tài liệu `vilo.pdf` mô tả dữ liệu dịch ở quy mô khoảng 100k câu song ngữ. Kiểm tra thực tế trong repository hiện tại cho thấy thư mục `dich-vi_lo/dataset/vi_lo_dataset_split` đã được chia sẵn thành ba tệp `train.csv`, `valid.csv` và `test.csv`, với hai cột `vi` và `lo`. Số lượng cụ thể như sau:

Bảng 2.7 Thống kê dữ liệu song ngữ Việt - Lào trong repository

Thành phần	Số câu
Train set	90.000
Validation set	5.000
Test set	5.000
Tổng số cặp câu	100.000

Quan sát nhanh trên các mẫu dữ liệu cho thấy tập song ngữ này khá đa miền: có câu hội thoại đời thường, câu kể, câu mô tả, tên riêng và câu văn dài. Đây là ưu điểm về độ phủ, nhưng đồng thời cũng tạo ra thách thức vì dữ liệu có thể chứa nhiều ở tên riêng, viết hoa, phiên âm hoặc các cấu trúc câu không đồng đều.



Hình 2.14 Tỷ lệ chia bộ dữ liệu song ngữ Việt – Lào

2.7.5 Tiền xử lý cho mô-đun dịch Việt - Lào

So với mô-đun ghép câu, tiền xử lý của mô-đun dịch tập trung nhiều hơn vào tokenizer đa ngôn ngữ của NLLB. Trong bản hiện thực hiện tại, các bước chính có thể tóm tắt như sau:

- dữ liệu đã được tách sẵn theo câu trong các tệp CSV;
- cột nguồn là tiếng Việt, cột đích là tiếng Lào;
- tokenizer của NLLB gán mã ngôn ngữ nguồn `vie_Latn` và ngôn ngữ đích `lao_Lao`;
- mô hình sử dụng token hóa kiểu subword của NLLB để giảm vấn đề từ ngoài từ điển;
- khi suy luận, mô hình được ép bắt đầu bằng token ngôn ngữ tiếng Lào để tránh sinh nhầm ngôn ngữ đích.

Trong thực nghiệm sau này, nếu cần nâng chất lượng mô hình, hai hướng tiền xử lý nên ưu tiên là: làm sạch tên riêng và thống nhất quy tắc viết hoa, dấu câu trên cặp dữ liệu song ngữ. Đây là hai nguồn nhiễu dễ gặp nhất trong tập dữ liệu quan sát được tại thời điểm viết báo cáo.

3. XÂY DỰNG PHẦN MỀM VÀ TÍCH HỢP HỆ THỐNG

3.1 Yêu cầu chức năng và phi chức năng

Trước khi đi vào cài đặt, hệ thống cần được đặc tả bằng các yêu cầu chức năng và phi chức năng. Việc đặc tả rõ ràng giúp liên hệ trực tiếp giữa mục tiêu nghiên cứu ở Chương 1 và phần hiện thực phần mềm ở Chương 3.

3.1.1 Yêu cầu chức năng

- Nhận video từ webcam theo thời gian thực.
- Nhận diện ký hiệu và hiển thị kết quả dự đoán.
- Lưu buffer từ vệt và ghép thành câu tiếng Việt.
- Dịch câu tiếng Việt sang tiếng Lào.
- Cho phép xóa buffer, xác nhận câu và thoát chương trình.

3.1.2 Yêu cầu phi chức năng

- Thời gian phản hồi đủ nhanh để demo trực tiếp.
- Hệ thống dễ mở rộng thêm lớp ký hiệu và thêm ngôn ngữ đích.
- Cấu trúc code tách mô-đun để dễ bảo trì.
- Có khả năng chạy ở chế độ giảm tải khi không muốn nạp mô hình dịch lớn.
- Có cơ chế fallback để tầng NLP vẫn sinh được câu tối thiểu khi mô hình sinh chưa sẵn sàng.

Các yêu cầu này dẫn đến lựa chọn thiết kế mô-đun hóa: nhận diện, ghép câu và dịch máy được giữ thành các khối độc lập, có thể kiểm thử riêng trước khi tích hợp. Đây cũng là lý do project có cả script kiểm thử dạng dòng lệnh và script realtime dùng webcam.

3.2 Phân tích use case

Phần use case giúp liên kết bài toán kỹ thuật với hành vi sử dụng thực tế của người dùng. Đối với đề tài này, actor chính là người vận hành hệ thống hoặc người dùng cần thực hiện nhận diện và dịch ngôn ngữ ký hiệu.

1. **Khởi động hệ thống và webcam:** người vận hành chạy chương trình realtime, hệ thống nạp MediaPipe, mô hình TFLite và các cấu hình cần thiết.

2. **Thực hiện ký hiệu:** người dùng đưa tay vào khung hình và thực hiện ký hiệu. Hệ thống trích xuất landmarks, duy trì bộ đệm frame và hiển thị top-k nhãn dự đoán.
3. **Xác nhận token:** khi nhãn hiện tại phù hợp, người vận hành thêm nhãn đó vào buffer token. Bước này giúp giảm tác động của các dự đoán nhiễu.
4. **Sinh câu và dịch:** khi buffer đã đủ nội dung, người vận hành yêu cầu hệ thống ghép thành câu tiếng Việt và dịch sang tiếng Lào nếu mô hình dịch đang bật.
5. **Xóa buffer hoặc bắt đầu lại:** nếu nhận diện sai hoặc người dùng muốn nhập câu mới, hệ thống cho phép xóa buffer token và reset bộ đệm frame.
6. **Kết thúc phiên làm việc:** người vận hành thoát chương trình, giải phóng camera và đóng cửa sổ hiển thị.

Luồng use case trên phản ánh đúng hiện trạng triển khai: hệ thống chưa tự động quyết định toàn bộ câu dài liên tục, mà có bước xác nhận token để tăng độ ổn định trong demo. Đây là lựa chọn phù hợp với giai đoạn nguyên mẫu vì người dùng vẫn kiểm soát được đầu vào của tầng NLP và tầng dịch.

3.3 Xây dựng mô-đun nhận diện ký hiệu

Nếu Chương 2 tập trung vào việc lý giải vì sao chọn MediaPipe Holistic và mô hình TFLite pre-trained (Kaggle ASL Signs), thì phần này mô tả cách mô-đun đã được hiện thực thành mã nguồn cụ thể. Ở snapshot hiện tại, module nhận diện được tổ chức gọn trong thư mục `external/google-asl-250/`, bao gồm file mô hình, bảng ánh xạ nhãn và script suy luận thời gian thực.

3.3.1 Cấu trúc mã nguồn của mô-đun

Bảng 3.1 Các file chính của mô-đun nhận diện

File	Vai trò
external/google-asl-250/ realtime_asl_250.py	Chương trình webcam thời gian thực: khởi tạo TFLite Interpreter, đọc camera, duy trì sliding window landmarks, suy luận mô hình và hiển thị top-k dự đoán.
external/google-asl-250/ model.tflite	Trọng số mô hình đã huấn luyện sẵn, giải pháp đạt giải nhất cuộc thi Kaggle ASL Signs. Kích thước ~ 11 MB.
external/google-asl-250/ sign_to_prediction_index_ map.json	Bảng ánh xạ 250 tên từ vựng ký hiệu sang chỉ số đầu ra tương ứng của mô hình.
external/google-asl-250/ MODEL_README.md	Tài liệu ghi chú nguồn gốc mô hình và giấy phép MIT.

Cấu trúc này rất gọn so với các mô-đun NLP: toàn bộ logic nhận diện được đóng gói trong một file Python duy nhất, kèm theo mô hình TFLite và bảng nhãn. Việc sử dụng mô hình pre-trained giúp giảm đáng kể số lượng file cần quản lý, không cần notebook huấn luyện hay checkpoint riêng.

3.3.2 Hiện thực trích xuất landmarks bằng MediaPipe

Lớp ASL250Recognizer trong realtime_asl_250.py khởi tạo MediaPipe Holistic với cấu hình:

- model_complexity=1;
- refine_face_landmarks=False;
- min_detection_confidence=0.5;
- min_tracking_confidence=0.5.

Hàm _extract_frame_landmarks(results) là điểm vào quan trọng nhất của tầng thị giác. Luồng xử lý gồm các bước:

1. lấy landmarks khuôn mặt (468 điểm), tay trái (21 điểm), pose (33 điểm) và tay phải (21 điểm);
2. mỗi nhóm landmarks được trải phẳng thành ma trận $(N, 3)$ với tọa độ (x, y, z) ;

3. nếu thiếu bất kỳ nhóm landmark nào (ví dụ tay không xuất hiện), thay bằng ma trận NaN cùng kích thước;
4. nối bốn nhóm theo thứ tự `face`, `left_hand`, `pose`, `right_hand` thành ma trận cuối cùng kích thước (543, 3).

Thiết kế “điền NaN khi thiếu landmark” là một chi tiết kỹ thuật quan trọng, khác biệt so với phương pháp điền zero thông thường. Giá trị NaN cho phép mô hình TFLite phân biệt rõ giữa “không có dữ liệu” và “tọa độ bằng 0”, từ đó tránh tạo nhiễu đầu vào không cần thiết.

Thứ tự nối landmarks (`face` → `left_hand` → `pose` → `right_hand`) tuân theo quy ước của cuộc thi Kaggle ASL Signs và phải được giữ nguyên để đảm bảo tương thích với mô hình pre-trained.

3.3.3 *Hiện thực nạp và suy luận mô hình TFLite*

Trong phương thức khởi tạo, mô hình TFLite được nạp bằng TensorFlow Lite Interpreter. Quá trình khởi tạo gồm:

- nạp file `model.tflite`;
- lưu lại chỉ số tensor đầu vào (`input_index`) và đầu ra (`output_index`);
- nạp bảng ánh xạ nhãn từ `sign_to_prediction_index_map.json`;
- tạo ánh xạ ngược `id_to_sign` để chuyển từ chỉ số sang tên từ vựng.

Phương thức `predict()` thực hiện suy luận theo các bước:

1. kiểm tra bộ đệm đã đủ tối thiểu 16 frame chưa;
2. gom toàn bộ frames trong bộ đệm thành tensor đầu vào;
3. gọi `resize_tensor_input()` để điều chỉnh kích thước đầu vào theo số frame thực tế;
4. gọi `allocate_tensors()`, `set_tensor()`;
5. gọi `invoke()` để chạy suy luận;
6. lấy kết quả từ `get_tensor()`;
7. áp dụng softmax nếu đầu ra chưa ở dạng xác suất;
8. trả về top-k nhãn có xác suất cao nhất.

Một chi tiết đáng chú ý là hàm `_softmax_if_needed()`: hàm này kiểm tra xem đầu ra của mô hình đã ở dạng xác suất (tổng ≈ 1 , tất cả ≥ 0) hay chưa. Nếu chưa, nó tự động áp dụng softmax. Thiết kế này giúp code hoạt động ổn định bất kể mô hình TFLite trả về logits hay xác suất.

3.3.4 *Hiện thực suy luận thời gian thực*

Trong phương thức `run()`, luồng chạy chính của mô-đun nhận diện có thể tóm tắt như sau:

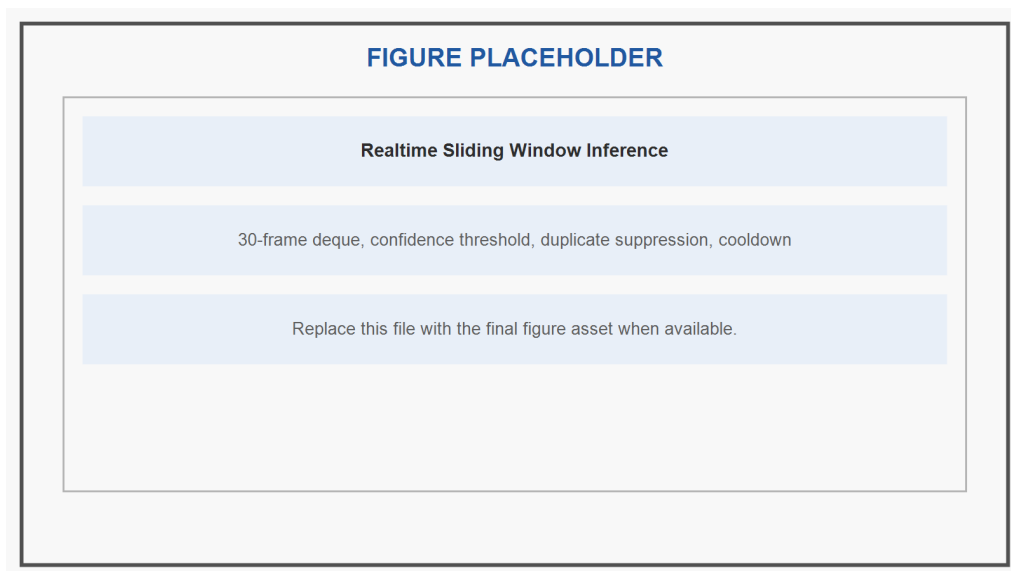
1. mở webcam với độ phân giải 960x540;
2. với mỗi frame: lật ảnh như gương, chuyển sang RGB và xử lý bằng MediaPipe Holistic;
3. chỉ gom landmarks vào bộ đệm khi phát hiện có bàn tay (trái hoặc phải);
4. mỗi 8 frame (`predict_every=8`), gọi `predict()` nếu bộ đệm đủ 16 frame;
5. hiển thị top-3 dự đoán trên giao diện video;
6. vẽ skeleton pose và bàn tay lên khung hình bằng MediaPipe drawing utilities.

Từ góc nhìn trải nghiệm người dùng, ba cơ chế nhỏ nhưng hiệu quả là:

- **chỉ gom khi có tay**: lọc các frame không có ý nghĩa ký hiệu;
- **predict_every = 8**: giảm tải tính toán, không cần suy luận mỗi frame;
- **min_frames = 16**: đảm bảo mô hình có đủ ngữ cảnh thời gian trước khi dự đoán.

Hai phím điều khiển chính:

- Space: reset bộ đệm và kết quả dự đoán;
- q: thoát chương trình.



Hình 3.1 Sơ đồ sliding window 16–60 frame và logic “chỉ gom khi có tay” trong suy luận realtime

3.3.5 Nhận xét dưới góc nhìn kỹ thuật phần mềm

Mô-đun nhận diện hiện tại có nhiều điểm làm tốt ở khía cạnh phần mềm:

- toàn bộ logic được đóng gói trong một lớp ASL250Recognizer duy nhất, dễ tích hợp và tái sử dụng;
- giao diện giữa mô-đun nhận diện và mô-đun NLP chỉ là nhãn gloss, nên dễ tích hợp;
- mô hình TFLite chạy trên CPU, không yêu cầu GPU, phù hợp với phần cứng phổ thông;
- code hỗ trợ nhiều camera backend (CAP_DSHOW trên Windows, CAP_ANY trên các hệ điều hành khác);
- có argparse để cấu hình camera, max-frames, min-frames từ dòng lệnh.

Bên cạnh đó cũng có vài giới hạn cần ghi nhận trung thực trong báo cáo:

- phần tích hợp với mô-đun NLP phía sau (ghép câu tiếng Việt, dịch Việt - Lào) chưa được nối trực tiếp trong script hiện tại; cần thêm bước ánh xạ gloss tiếng Anh sang token tiếng Việt;
- vì là mô hình pre-trained, nhóm không thể can thiệp vào kiến trúc bên trong hoặc tinh chỉnh thêm cho VSL;
- cấu hình hiện tại nhắm tới demo một người, một webcam, chưa phải bài toán nhiều người đồng thời hoặc bối cảnh nền phức tạp.

Nhìn chung, ở mức đồ án môn học, mô-đun này đã đạt một ngưỡng hiện thực hóa đầy đủ: có mô hình pre-trained chất lượng cao, có script suy luận realtime và có cổng nối lên tầng NLP thông qua đầu ra gloss.

3.4 Xây dựng mô-đun ghép từ thành câu tiếng Việt

Nếu Chương 2 tập trung vào việc giải thích vì sao chọn kiến trúc lai Rule-based, BARTpho và fallback, thì phần này trình bày cách mô-đun được hiện thực hóa trong mã nguồn và cách nó tham gia vào pipeline phần mềm tổng thể. Điểm quan trọng nhất là project không coi bài toán ghép câu như một khối “black-box”, mà tách rõ thành nhiều lớp xử lý nhỏ để dễ kiểm thử và dễ tích hợp.

3.4.1 Các thành phần chính trong mã nguồn

Mô-đun ghép câu được tổ chức xoay quanh bốn thành phần:

- SentenceBuilder: điều phối toàn bộ quá trình nhận chuỗi gloss, chọn đường xử lý phù hợp và trả về câu hoàn chỉnh.
- sentence_rules.json: tập luật cứng cho các mẫu câu thường gặp, có tính xác định cao.
- BartphoSentencePredictor: nạp tokenizer, mô hình đã fine-tune và thực hiện suy luận sinh câu.
- fallback_corrector: ánh xạ các token cơ bản sang tiếng Việt và hoàn thiện định dạng câu khi mô hình chưa sẵn sàng.

Bảng 3.2 Vai trò của các file quan trọng trong mô-đun ghép câu

File	Chức năng
src/nlp/sentence_builder.py	Điều phối thứ tự ưu tiên rule, sau đó BARTpho, rồi fallback.
src/nlp/bartpho_predictor.py	Nạp mô hình đã fine-tune và sinh câu tiếng Việt bằng beam search.
src/nlp/fallback_corrector.py	Chuẩn hóa token, ánh xạ từ thô và thêm viết hoa, dấu câu.
configs/sentence_rules.json	Lưu các mẫu câu tần suất cao để phản hồi nhanh và ổn định.
configs/nlp_config.json	Lưu cấu hình huấn luyện và suy luận của BARTpho.

outputs/reports/bartpho_sentence_reports		
Name	Type	Size
bartpho_sentence_metrics_full.json	JSON	456 B
bartpho_sentence_metrics_summary_full.csv	CSV	576 B
bartpho_sentence_predictions_full.csv	CSV	322 KB
bartpho_sentence_report_text.txt	TXT	2 KB
bartpho_sentence_sample_predictions_full.txt	TXT	35 KB
bartpho_sentence_reports.zip	ZIP	57 KB

Hình 3.2 Các file báo cáo được sinh sau khi đánh giá mô hình

3.4.2 Luồng xử lý chính

Trong lớp SentenceBuilder, đầu vào trước hết được chuẩn hóa bằng cách loại bỏ khoảng trắng thừa. Sau đó hệ thống xử lý theo thứ tự ưu tiên:

1. nếu chuỗi đầu vào rỗng, trả về chuỗi rỗng;
2. nếu chuỗi đầu vào khớp đúng một rule trong `sentence_rules.json`, trả về câu rule tương ứng;
3. nếu BARTpho đã được nạp thành công, gọi hàm `predict()` để sinh câu;
4. nếu mô hình chưa có hoặc nạp lỗi, gọi `normalize_raw_text()` và `to_sentence()` để rơi về chế độ dự phòng.

Thứ tự này có ý nghĩa thực tiễn rất rõ:

- rule-based đảm bảo các câu giao tiếp quan trọng như xin chào, xin lỗi, yêu cầu hỗ trợ luôn cho kết quả ổn định;
- BARTpho đảm nhiệm những trường hợp chuỗi gloss dài hơn hoặc linh hoạt hơn;
- fallback giúp hệ thống không bị “gãy” khi thiếu mô hình huấn luyện.



Hình 3.3 Lưu đồ xử lý chuỗi gloss trong mô-đun ghép câu

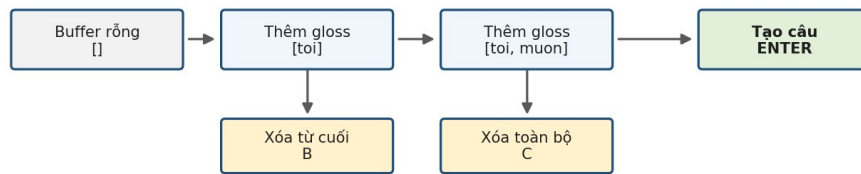
3.4.3 Hiện thực tầng rule-based và fallback

Tầng rule-based được hiện thực bằng một tệp JSON đơn giản, trong đó mỗi khóa là một chuỗi gloss chuẩn hóa, còn giá trị là câu tiếng Việt đích. Cách làm này tuy đơn giản nhưng rất hiệu quả với các mẫu câu ngắn và các cụm ý nghĩa gần như cố định trong miền giao tiếp hỗ trợ, ví dụ:

- xin_chao cho ra câu *Xin chào*.
- toi can giup_do cho ra câu *Tôi cần giúp đỡ*.
- toi muon uong_nuoc cho ra câu *Tôi muốn uống nước*.

Trong trường hợp không có mô hình sinh câu, file `fallback_corrector.py` dùng một bảng ánh xạ token như `toi` thành `tôi`, `uong_nuoc` thành `uống nước`, `bac_si` thành `bác sĩ`. Sau khi ghép các từ lại, hệ thống tự viết hoa ký tự đầu và thêm dấu chấm nếu cuối câu chưa có dấu kết thúc. Nhờ đó, ngay cả khi chỉ chạy ở chế độ dự phòng, đầu ra vẫn đọc được và đủ dùng trong các demo tối thiểu.

Cơ chế trạng thái của SentenceBuffer



Hình 3.4 Cơ chế trạng thái của SentenceBuffer khi thêm, xóa và tạo câu

3.4.4 Hiện thực suy luận bằng BARTpho

Lớp `BartphoSentencePredictor` chịu trách nhiệm nạp mô hình từ thư mục `models/bartpho_sentence`. Quá trình suy luận gồm các bước:

- chuẩn hóa lại chuỗi đầu vào;
- thêm tiền tố nhiệm vụ `chuyen_cau_ky_hieu::`;
- token hóa đầu vào với độ dài tối đa 64 token;
- chuyển tensor sang cuda nếu có GPU, ngược lại dùng CPU;
- sinh câu bằng `model.generate()` với `num_beams = 4`;
- giải mã chuỗi token sinh ra thành câu tiếng Việt hoàn chỉnh.

Điểm đáng chú ý là project giữ hoàn toàn nhất quán giữa huấn luyện và suy luận. File `train_bartpho.py` cũng dùng chính tiền tố `chuyen_cau_ky_hieu::`, cùng độ dài tối đa và cùng logic token hóa. Điều này giảm nguy cơ lệch phân phối dữ liệu giữa train và inference.

```
Kiểm thử CLI - BARTphoPredictor

(.venv) PS F:\VSL_Vietnamese_NLP> python -m src.nlp.bartpho_predictor --text "toi muon uong_nuoc"
Gloss: toi muon uong_nuoc
Sentence: Tôi muốn uống nước.

(.venv) PS F:\VSL_Vietnamese_NLP> python -m src.nlp.bartpho_predictor --text "toi can giup_do"
Gloss: toi can giup_do
Sentence: Tôi cần giúp đỡ.
```

Hình 3.5 Kiểm thử suy luận BARTpho bằng chuỗi gloss đầu vào

3.4.5 Khả năng tích hợp với pipeline realtime

Mô-đun ghép câu được thiết kế để dễ tích hợp vào hệ thống thời gian thực:

- giao diện đầu vào chỉ cần một chuỗi gloss đã chuẩn hóa;
- đầu ra là chuỗi văn bản thuần, có thể đưa tiếp sang mô-đun dịch hoặc hiển thị trực tiếp lên giao diện;
- có cơ chế suy giảm chất lượng mềm khi thiếu mô hình, thay vì dừng chương trình;
- logic được tách lớp rõ ràng, thuận tiện cho việc viết test riêng cho rule, model và fallback.

Về mặt phần mềm, đây là một quyết định quan trọng. Trong một pipeline nhiều mô-đun, tầng NLP phải vừa đủ “thông minh” để cải thiện đầu ra, vừa đủ “chắc” để không phá vỡ toàn bộ hệ thống khi môi trường triển khai chưa đầy đủ. Kiến trúc hiện tại đáp ứng tốt yêu cầu đó và là nền tảng phù hợp để sau này nối trực tiếp với mô-đun nhận diện và mô-đun dịch Việt - Lào.

3.5 Xây dựng mô-đun dịch Việt - Lào

Sau khi mô-đun ghép câu trả về câu tiếng Việt hoàn chỉnh, mô-đun dịch Việt - Lào tiếp nhận chuỗi đó và thực hiện suy luận dịch máy. So với mô-đun ghép câu, phần cài đặt của mô-đun dịch ngắn hơn, nhưng mỗi bước đều phụ thuộc chặt vào cấu hình của mô hình nền đa ngôn ngữ và adapter LoRA.

3.5.1 Các thành phần chính trong mã nguồn

Trong snapshot hiện tại, mô-đun dịch được tổ chức khá gọn, trọng tâm nằm ở tệp `translate.py`. Tệp này chịu trách nhiệm:

- nạp tokenizer từ thư mục adapter;
- nạp mô hình nền facebook/nllb-200-distilled-600M;
- ghép adapter LoRA bằng `PeftModel.from_pretrained()`;
- cung cấp hàm `translate_vi_to_lao(text)` để suy luận;
- chạy giao diện dòng lệnh tối giản để thử dịch từng câu.

Bảng 3.3 Các thành phần phần mềm chính của mô-đun dịch Việt - Lào

Thành phần	Chức năng
<code>translate.py</code>	Khởi tạo tokenizer, mô hình nền, adapter LoRA và thực hiện suy luận dịch.
<code>model/.../adapter_config.json</code>	Lưu cấu hình LoRA như hạng thấp, dropout và các lớp mục tiêu.
<code>model/.../adapter_model.safetensors</code>	Chứa trọng số adapter đã được fine-tune cho cặp Việt – Lào.
<code>dataset/vi_lo_dataset_split</code>	Lưu các tập train, validation, test ở dạng CSV.

3.5.2 Quy trình khởi tạo mô hình

Khi chương trình bắt đầu, `translate.py` thực hiện ba bước nạp chính:

1. nạp tokenizer từ thư mục adapter cục bộ;
2. nạp mô hình nền NLLB-200 Distilled 600M;
3. gắn adapter LoRA vào mô hình nền bằng PEFT.

Việc tách mô hình thành *base model* và *adapter* có hai lợi ích rõ rệt:

- giảm kích thước phần cần lưu trữ trong repository;
- cho phép thay adapter theo từng tác vụ mà không phải sao chép toàn bộ mô hình dịch nền.

Về mặt triển khai, đây là một kiến trúc rất phù hợp cho đề án vì có thể tái sử dụng cùng một mô hình nền NLLB cho nhiều tác vụ hoặc nhiều cặp ngôn ngữ khác nhau trong tương lai.

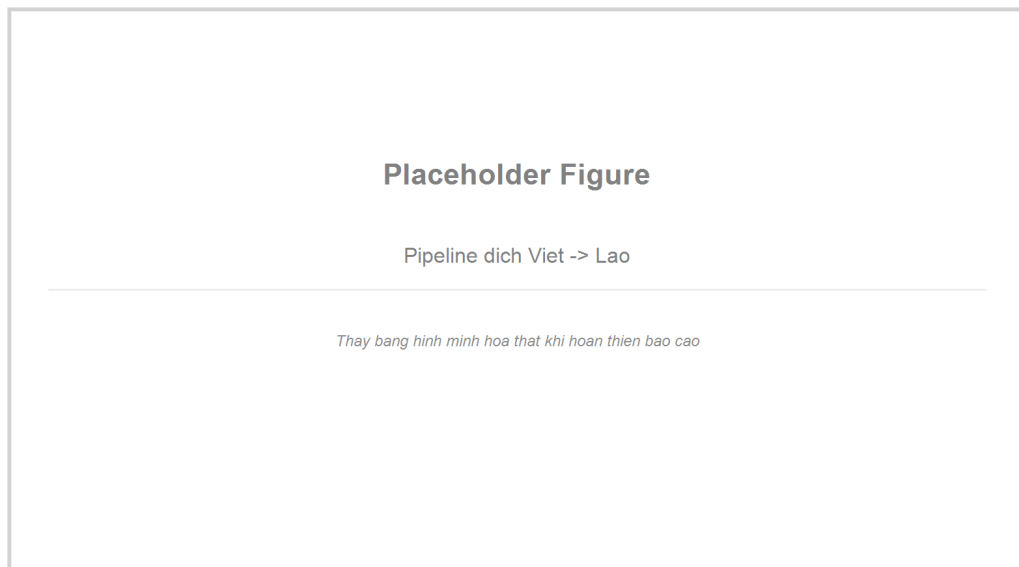
3.5.3 Quy trình suy luận một câu dịch

Hàm `translate_vi_to_lao(text)` hiện thực hóa suy luận theo đúng logic của NLLB:

- đặt `tokenizer.src_lang = "vie_Latn"`;
- mã hóa câu đầu vào thành tensor;
- gọi `model.generate()` với `forced_bos_token_id` tương ứng `lao_Lao`;

- giới hạn độ dài sinh ở 128 token;
- dùng `num_beams = 5` để tăng độ ổn định của đầu ra;
- giải mã tensor sinh ra thành câu tiếng Lào.

Ý nghĩa của cấu hình này là mô hình luôn biết rõ ngôn ngữ nguồn và ngôn ngữ đích trong quá trình suy luận. Với các mô hình dịch đa ngôn ngữ, đây là điều bắt buộc, vì thiếu token ngôn ngữ sẽ dễ dẫn đến đầu ra sai ngôn ngữ hoặc pha trộn mã ngôn ngữ không mong muốn.



Hình 3.6 Luồng cài đặt và suy luận trong mô-đun dịch Việt – Lào

3.5.4 Vai trò của LoRA trong phần mềm

Mặc dù phần mã gọi LoRA chỉ gồm vài dòng, adapter thực chất là thành phần quan trọng nhất quyết định việc mô hình nền NLLB có thích nghi được với cặp Việt - Lào hay không. Trong project này, LoRA được gắn vào các lớp attention projection và lớp đầu ra, tức là đúng những điểm có ảnh hưởng mạnh đến khả năng căn chỉnh biểu diễn giữa câu nguồn và câu đích.

So với full fine-tuning, cách làm này mang lại ba lợi ích trực tiếp cho phần mềm:

- dễ đóng gói mô hình hơn;
- ít rủi ro hơn khi cập nhật mô hình nền;
- thuận tiện cho việc thử nhiều adapter khác nhau trên cùng một interface dịch.

3.5.5 Tích hợp với mô-đun ghép câu phía trước

Mô-đun dịch chỉ thực sự phát huy hiệu quả khi câu tiếng Việt đầu vào đã đủ mạch lạc. Vì vậy, chất lượng của mô-đun này phụ thuộc mạnh vào đầu ra của mô-đun ghép câu:

- nếu câu tiếng Việt rõ chủ-vị và đúng ý nghĩa, NLLB sẽ dịch chính xác hơn;
- nếu câu tiếng Việt còn thiếu thành phần, dịch máy dễ tạo ra câu tiếng Lào khó hiểu hoặc lệch nghĩa;
- nếu tầng ghép câu đã chuẩn hóa dấu câu và viết hoa, tokenizer của mô hình dịch cũng làm việc ổn định hơn.

Nói cách khác, hai mô-đun NLP trong hệ thống không tách rời, mà hỗ trợ cho nhau theo kiểu nối tầng: mô-đun ghép câu giảm nhiễu ngôn ngữ nội bộ, còn mô-đun dịch chuyển ý nghĩa đó sang ngôn ngữ đích.

3.5.6 Nhận xét về hiện thực phần mềm

Điểm mạnh của phần cài đặt hiện tại là đơn giản, rõ ràng và đúng với thực tế nghiên cứu ứng dụng. Chỉ với một script ngắn, hệ thống đã có thể nạp được mô hình nền đa ngôn ngữ, adapter tinh chỉnh riêng cho tác vụ và thực hiện suy luận câu mới.

Tuy nhiên, vẫn còn các điểm cần hoàn thiện thêm trong giai đoạn sau:

- chưa có script đánh giá full test set đi kèm trực tiếp trong snapshot hiện tại;
- chưa có lớp giao tiếp trung gian để gọi mô-đun dịch như một service trong ứng dụng lớn hơn;
- lần chạy đầu tiên có thể phụ thuộc vào việc mô hình nền đã có cache cục bộ hay chưa.

Những điểm này không làm sai kiến trúc của mô-đun, nhưng là các hạng mục nên được bổ sung nếu muốn biến phần nghiên cứu thành sản phẩm phần mềm hoàn chỉnh hơn.

3.6 Tích hợp pipeline thời gian thực

Sau khi từng mô-đun hoạt động độc lập, bước tiếp theo là ghép chúng thành một pipeline thời gian thực. Trong repository hiện tại, script `realtime_integrated.py` đảm nhiệm vai trò điều phối: gọi mô-đun nhận diện theo luồng webcam, quản lý token buffer, gọi mô-đun ghép câu tiếng Việt và tùy chọn gọi mô-đun dịch Việt – Lào.

Vòng lặp realtime gồm các bước chính:

1. đọc frame từ webcam và đưa qua mô-đun nhận diện ASL-250;
2. hiển thị top-k dự đoán hiện tại để người vận hành quan sát;
3. khi nhấn a, lấy top-1 prediction hiện tại, ánh xạ sang token tiếng Việt và thêm vào buffer;
4. khi nhấn b, gửi buffer sang SentenceBuilder để sinh câu tiếng Việt, sau đó dịch sang tiếng Lào nếu không chạy ở chế độ `-no-translation`;
5. khi nhấn c, xóa token buffer để bắt đầu câu mới;
6. khi nhấn SPACE, reset bộ đệm frame của mô-đun nhận diện;
7. khi nhấn q, thoát chương trình và giải phóng camera.

Thiết kế này có ưu điểm là đơn giản, dễ quan sát và phù hợp cho demo học thuật. Người vận hành có thể kiểm soát thời điểm thêm token, thời điểm ghép câu và thời điểm dịch, nhờ đó giảm rủi ro do mô hình nhận diện nhảy nhót trong vài frame ngắn. Khi mô hình dịch chưa được tải hoặc máy không đủ tài nguyên, tùy chọn `-no-translation` giúp kiểm thử hai tầng đầu mà không làm gián đoạn toàn bộ pipeline.

3.7 Giao diện chạy thử và các đoạn mã tiêu biểu

Phần giao diện chạy thử tập trung vào tính quan sát được của pipeline. Khi chương trình realtime hoạt động, màn hình cần thể hiện tối thiểu ba nhóm thông tin: khung hình webcam kèm skeleton/landmarks, top-k nhãn nhận diện hiện tại và trạng thái buffer/câu đầu ra. Cách hiển thị này giúp người vận hành biết mô hình đang nhìn thấy gì, đang dự đoán nhãn nào và dữ liệu nào sẽ được đưa sang tầng NLP.

- Khung video hiển thị người dùng, skeleton pose và landmarks bàn tay do MediaPipe vẽ.
- Vùng dự đoán hiển thị top-3 nhãn ký hiệu cùng xác suất tương ứng.
- Vùng buffer hiển thị các token đã được xác nhận bằng phím a.
- Vùng kết quả hiển thị câu tiếng Việt sau khi nhấn b và câu tiếng Lào nếu bật mô-đun dịch.

Các đoạn mã nên đưa vào báo cáo chỉ cần đại diện cho logic chính, chẳng hạn hàm ánh xạ nhãn ASL sang token tiếng Việt, thao tác thêm token vào buffer và lời gọi ghép câu/dịch. Phần code dài như khởi tạo MediaPipe, nạp TensorFlow Lite Interpreter hoặc cấu hình NLLB nên được mô tả bằng lời trong thân báo cáo để tránh làm loãng nội dung kỹ thuật chính.

4. THỰC NGHIỆM, ĐÁNH GIÁ, KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

4.1 Thiết lập thực nghiệm

Thiết lập thực nghiệm cần được mô tả rõ để các kết quả đánh giá được hiểu đúng bối cảnh. Do hệ thống gồm ba mô-đun có bản chất khác nhau, phần đánh giá được tách thành bốn nhóm: nhận diện ký hiệu, khôi phục câu tiếng Việt, dịch Việt – Lào và đánh giá tích hợp end-to-end.

Trong snapshot hiện tại, mô-đun nhận diện sử dụng mô hình TFLite pre-trained nên không có log huấn luyện riêng trong repository. Nhóm bổ sung script đánh giá riêng để đo latency và tính Top-k Accuracy, Precision, Recall, F1 khi có tập landmarks/video gán nhãn; tuy nhiên workspace hiện chưa có test manifest gán nhãn cho tầng nhận diện nên không công bố số accuracy/F1 tự suy diễn. Mô-đun khôi phục câu tiếng Việt có báo cáo đánh giá trên tập test của bộ dữ liệu VSL Vietnamese NLP. Mô-đun dịch Việt – Lào có dữ liệu train/validation/test và adapter LoRA; các chỉ số Train loss, Validation loss và BLEU được trích từ tài liệu `vilo.pdf`. Các metric chrF và TER chưa thấy số liệu truy vết trong workspace nên không được tự suy diễn.

Bảng 4.1 Môi trường và dữ liệu dùng trong thực nghiệm

Thành phần	Mô tả
Thiết bị demo	Máy tính Windows có webcam; mô-đun nhận diện chạy bằng TensorFlow Lite/MediaPipe trên CPU trong chế độ demo.
Mô-đun nhận diện	Mô hình <code>model.tflite</code> từ Kaggle ASL Signs, 250 lớp, đầu vào 543 landmarks x 3 tọa độ, cửa sổ 16–60 frame.
Mô-đun tiếng Việt	BARTpho kết hợp rule/fallback; dữ liệu chính gồm 32.755 cặp gloss – câu tiếng Việt, chia train/validation/test.
Mô-đun dịch	NLLB-200 Distilled 600M kết hợp adapter LoRA; dữ liệu song ngữ Việt – Lào gồm 100.000 cặp câu, chia 90.000/5.000/5.000.
Thư viện chính	Python, OpenCV, MediaPipe, TensorFlow Lite, PyTorch, Transformers, PEFT, SentencePiece.

Để tránh trộn lẫn kết quả giữa các tầng, mỗi mô-đun được đánh giá theo đúng đầu vào chuẩn của nó. Tầng nhận diện được đánh giá bằng latency TFLite và quan sát webcam; Top-k Accuracy, Macro F1 và confusion matrix được xác định là metric bắt buộc khi có tập video/landmark gán nhãn riêng. Tầng tiếng Việt được đánh giá bằng loss, perplexity, Exact Match, BLEU, ROUGE-L, chrF và Token F1. Tầng dịch được đề xuất

đánh giá bằng BLEU, chrF, TER và kiểm tra thủ công về adequacy/fluency. Đánh giá end-to-end tập trung vào độ đúng của câu cuối cùng, độ trễ và khả năng vận hành ổn định khi chạy liên tục.

4.2 Đánh giá mô hình nhận diện ký hiệu

Mô-đun nhận diện là tầng đầu tiên của toàn bộ pipeline nên mọi sai số ở đây đều có thể lan xuống mô-đun ghép câu và mô-đun dịch. Vì mô hình hiện tại là một mô hình pre-trained (TFLite từ cuộc thi Kaggle ASL Signs), cách đánh giá sẽ khác so với một mô hình tự huấn luyện: nhóm không có quá trình training/validation thông thường, mà tập trung vào đánh giá chất lượng suy luận thực tế.

4.2.1 Các metric có thể đánh giá

Mặc dù mô hình nhận diện trong đề tài là mô hình pre-trained, mô-đun này vẫn cần được đánh giá như một bộ phân loại đa lớp. Điểm khác biệt là nhóm không báo cáo loss huấn luyện hay validation loss do không huấn luyện lại mô hình; thay vào đó, phần đánh giá tập trung vào chất lượng dự đoán trên tập kiểm thử gán nhãn và tốc độ suy luận. Các metric phù hợp bao gồm:

- **Top-1 Accuracy:** tỉ lệ dự đoán đúng nhãn ký hiệu trên tập kiểm thử;
- **Top-3/Top-5 Accuracy:** tỉ lệ nhãn đúng nằm trong top-k dự đoán;
- **Macro Precision, Macro Recall, Macro F1:** đánh giá cân bằng theo từng lớp, tránh việc một vài lớp xuất hiện nhiều làm sai lệch kết quả chung;
- **Weighted F1:** F1 có trọng số theo số mẫu của từng lớp;
- **Confusion matrix:** ma trận nhầm lẫn giữa các lớp ký hiệu;
- **Average inference time:** thời gian suy luận trung bình trên mỗi chuỗi landmarks;
- **FPS:** số khung hình xử lý được trong một giây, quan trọng cho demo realtime;
- **Đánh giá định tính:** quan sát trực tiếp khi sử dụng webcam.

Bảng 4.2 Ý nghĩa của các metric cho mô-đun nhận diện

Metric	Ý nghĩa	Cách diễn giải
Top-1 Accuracy	Tỉ lệ mẫu có lớp dự đoán cao nhất trùng đúng nhãn thật.	Càng cao càng tốt; là chỉ số chính của bài toán phân loại.
Top-3 Accuracy	Tỉ lệ mẫu mà nhãn thật xuất hiện trong 3 dự đoán cao nhất.	Hữu ích khi số lớp lớn và nhiều dấu hiệu gần nhau.
Top-5 Accuracy	Tỉ lệ mẫu mà nhãn thật xuất hiện trong 5 dự đoán cao nhất.	Cho thấy mô hình có “nhìn ra” vùng ứng viên đúng hay không.
Macro F1	Trung bình F1 theo từng lớp ký hiệu.	Phù hợp khi muốn xem mô hình có công bằng giữa các lớp hay không.
Confusion matrix	Bảng đếm nhãn thật và nhãn dự đoán.	Dùng để phát hiện các cặp ký hiệu dễ bị nhầm.
Latency / FPS	Tốc độ suy luận trên mỗi mẫu hoặc theo khung hình.	Quyết định khả năng dùng trong real-time demo.

Với tập kiểm thử gồm N mẫu, nhãn thật y_i và danh sách dự đoán top-k là $\hat{Y}_{i,k}$, các chỉ số Top-k Accuracy được tính như sau:

$$TopK = \frac{1}{N} \sum_{i=1}^N \mathbf{1}(y_i \in \hat{Y}_{i,k})$$

Precision, Recall và F1 theo từng lớp được tính từ số lượng true positive (TP), false positive (FP) và false negative (FN):

$$Precision = \frac{TP}{TP + FP}, \quad Recall = \frac{TP}{TP + FN}, \quad F1 = \frac{2 \times Precision \times Recall}{Precision + Recall}$$

4.2.2 Bối cảnh mô hình pre-trained

Cần nhấn mạnh rằng mô hình TFLite trong dự án là giải pháp đạt giải nhất cuộc thi Kaggle Google – Isolated Sign Language Recognition. Cuộc thi này có hơn 1.000 đội tham gia và đánh giá trên dữ liệu kiểm thử chính thức của Kaggle. Vì vậy, chất lượng của mô hình đã được kiểm chứng ở quy mô cạnh tranh quốc tế.

Tuy nhiên, kết quả trong cuộc thi Kaggle được đo trên dữ liệu landmarks đã chuẩn bị sẵn, trong khi pipeline thực tế của dự án phải trích xuất landmarks từ webcam theo thời gian thực. Điều này có thể tạo ra chênh lệch giữa chất lượng offline (trên dữ liệu Kaggle) và chất lượng online (trên webcam thực tế), do:

- chất lượng camera, ánh sáng và nền ảnh hưởng đến độ chính xác của MediaPipe;

- tốc độ ký hiệu và khoảng cách tới camera có thể khác so với điều kiện trong dữ liệu huấn luyện;
- một số từ vựng trong 250 lớp có thể gần nhau về hình thức, dễ gây nhầm.

Bảng 4.3 Thông tin tham chiếu từ cuộc thi Kaggle ASL Signs

Thuộc tính	Giá trị
Tên cuộc thi	Google – Isolated Sign Language Recognition
Số lớp từ vựng	250
Nguồn mô hình	Giải pháp đạt giải nhất (1st place solution)
Định dạng	TensorFlow Lite (.tflite)
Kích thước mô hình	~11 MB

4.2.3 Công cụ đánh giá định lượng

Để phần nhận diện không chỉ dừng ở mức “chạy mô hình pre-trained”, nhóm bổ sung script `external/google-asl-250/evaluate_asl_250.py`. Script này có hai chế độ:

- **benchmark-only**: đo thời gian suy luận của mô hình TFLite trên tensor landmarks giả lập có đúng kích thước đầu vào $N \times 543 \times 3$;
- **manifest evaluation**: đọc file CSV gồm hai cột `path, label`, chạy suy luận trên các file landmarks `.npy/.npz`, sau đó xuất Top-1, Top-3, Top-5 Accuracy, Macro Precision, Macro Recall, Macro F1, Weighted F1, latency và confusion matrix.

Lệnh benchmark tốc độ đã dùng trong snapshot này:

```
venv\Scripts\python.exe external\google-asl-250\evaluate_asl_250.py ^
--benchmark-only --runs 30 --warmup 5
```

Khi có tập kiểm thử landmarks gán nhãn, lệnh đánh giá phân loại có dạng:

```
venv\Scripts\python.exe external\google-asl-250\evaluate_asl_250.py ^
--manifest test_manifest.csv --output-dir external\google-asl-250\eval_resu
```

Bảng 4.4 Kết quả benchmark suy luận TFLite của mô-đun nhận diện

Độ dài chuỗi	Mean latency (ms)	P95 latency (ms)	FPS tương đương	
16 frame	13,26	16,57	75,40	
32 frame	26,26	36,91	38,09	
60 frame	44,92	49,92	22,26	

Các số liệu trong Bảng 4.4 chỉ đo phần suy luận TFLite trên CPU, chưa bao gồm thời gian đọc webcam, tiền xử lý ảnh và trích xuất landmarks bằng MediaPipe. Tuy vậy, kết quả này cho thấy riêng mô hình phân loại đủ nhanh để phục vụ demo realtime, vì trong code hiện tại mô hình chỉ được gọi mỗi 8 frame thay vì gọi ở mọi frame.

Bảng 4.5 Trạng thái các metric phân loại của mô-đun nhận diện trong snapshot hiện tại

Metric	Trạng thái	Ghi chú
Top-1 Accuracy	Chưa công bố số	Cần tập kiểm thử landmarks/video có nhãn thật. Repository hiện chỉ có mô hình pre-trained và label map, chưa có test manifest cho recognition.
Top-3 / Top-5 Accuracy	Chưa công bố số	Script đã hỗ trợ tính tự động khi có dữ liệu gán nhãn.
Macro Precision / Recall / F1	Chưa công bố số	Không thể tính đúng nếu thiếu y_true. Việc tự đưa số khi chưa có tập test sẽ không có giá trị học thuật.
Confusion matrix	Chưa công bố bảng	Sẽ được xuất ra file <code>recognition_confusion_matrix.csv</code> khi chạy đánh giá với manifest gán nhãn.
Latency / FPS	Đã đo	Có số liệu benchmark TFLite trong Bảng 4.4.

4.2.4 Đánh giá thực nghiệm qua demo realtime

Vì snapshot hiện tại chưa có tập test nhận diện riêng, phần đánh giá thực nghiệm qua webcam được dùng để kiểm tra khả năng vận hành thực tế của pipeline. Đây không thay thế cho Top-k Accuracy hay F1, nhưng có giá trị bổ sung vì pipeline thật phải đi qua camera, MediaPipe và cửa sổ trượt thời gian. Các tiêu chí đánh giá bao gồm:

- mô hình có nhận đúng các từ vựng phổ biến như hello, drink, food, please, thankyou hay không;
- mô hình phản hồi nhanh bao lâu sau khi người dùng bắt đầu ký hiệu;
- mô hình có ổn định (không nhảy liên tục giữa các nhãn) khi người dùng giữ tay trong tư thế;
- mô hình xử lý thế nào khi người dùng chuyển giữa các ký hiệu liên tiếp.

Bảng 4.6 Đánh giá định tính mô-đun nhận diện qua webcam

Kịch bản thử nghiệm	Kết quả	Nhận xét
Ký hiệu từ phổ biến như hello, drink	Hoạt động được trong demo	Các nhãn phổ biến có thể xuất hiện ổn định hơn khi tay nằm rõ trong khung hình và ánh sáng đủ.
Ký hiệu liên tiếp nhiều từ	Cần xác nhận từng token	Phiên bản hiện tại phù hợp hơn với cơ chế chọn token/top-1 vào buffer, chưa phải nhận diện câu liên tục hoàn toàn tự động.
Điều kiện ánh sáng yếu	Dễ giảm ổn định	MediaPipe có thể mất landmarks tay hoặc pose, kéo theo dự đoán TFLite kém tin cậy.
Tốc độ ký hiệu nhanh	Có thể bỏ sót động tác	Cửa sổ trượt cần đủ tối thiểu 16 frame; thao tác quá nhanh làm chuỗi landmarks thiếu ngữ cảnh.
Khoảng cách xa camera	Dễ mất chi tiết bàn tay	Khi bàn tay quá nhỏ hoặc bị che, landmarks không ổn định và xác suất dự đoán dao động mạnh.

4.2.5 Đánh giá định tính và phân tích lỗi

Bên cạnh các quan sát trực tiếp, phần nhận diện cần có thêm phân tích lỗi theo các hướng:

- **nhầm lớp do động tác gần giống:** nhiều từ khác nhau chỉ lệch hướng cổ tay hoặc biên độ chuyển động;
- **nhầm do điều kiện ghi hình:** ánh sáng yếu, nền lộn xộn, tay che nhau hoặc ra ngoài khung hình;
- **nhầm do sự khác biệt signer:** mô hình được huấn luyện trên signer trong dữ liệu Kaggle, có thể chưa khái quát tốt cho người dùng thực tế;
- **giới hạn 250 từ:** nếu người dùng thực hiện ký hiệu không nằm trong 250 lớp, mô hình sẽ dự đoán sai.

4.2.6 Lưu ý khi diễn giải kết quả

Ngay cả khi mô hình nhận diện hoạt động tốt, vẫn cần nhấn mạnh rằng:

- đây mới là chất lượng ở tầng phân loại gloss, chưa phải chất lượng câu tiếng Việt hay tiếng Lào cuối cùng;

- mô hình được huấn luyện trên dữ liệu ASL, nên kết quả tốt không đồng nghĩa mô hình đã “hiểu” trọn vẹn VSL thật trong thực địa;
- cần tách bạch rõ *recognition accuracy*, *sentence quality* và *translation quality*, thay vì gộp chung thành một đánh giá mơ hồ.

Trình bày như vậy vừa chính xác về mặt học thuật, vừa giúp người đọc hiểu đúng đóng góp thật của mô hình nhận diện trong toàn bộ hệ thống.

4.3 Đánh giá mô hình khôi phục câu tiếng Việt

Mô-đun ghép câu tiếng Việt là một bài toán sinh văn bản có cấu trúc. Vì vậy, nếu chỉ dùng một metric duy nhất sẽ khó phản ánh đầy đủ chất lượng đầu ra. Trong báo cáo này, phần đánh giá cần kết hợp cả *metric huấn luyện* và *metric tương đồng văn bản*:

- **Loss** và **Perplexity** phản ánh mô hình học phân phối token tốt đến mức nào;
- **Exact Match** đo tỉ lệ câu dự đoán trùng hoàn toàn với câu tham chiếu;
- **BLEU** đo mức trùng khớp n-gram sửa đổi giữa câu sinh ra và câu chuẩn;
- **ROUGE-L** nhấn mạnh chuỗi con chung dài nhất, phù hợp khi muốn nhìn cấu trúc câu;
- **chrF** đo mức tương đồng ở cấp ký tự, hữu ích với tiếng Việt vì còn liên quan tới dấu và biến thể chính tả;
- **Token F1** và **Similarity Ratio** giúp nhìn thêm ở mức từ và mức chuỗi tổng quát.

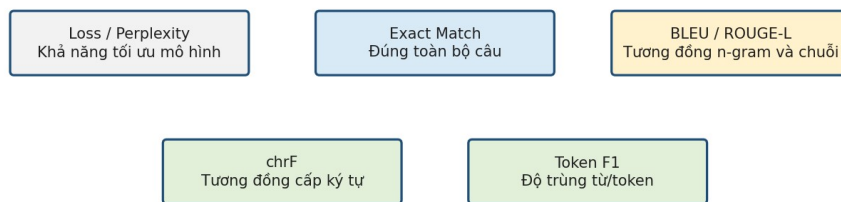
Việc dùng đồng thời các metric trên là phù hợp với các nghiên cứu *Gloss-to-Text* gần đây, nơi người ta thường không chỉ báo cáo BLEU mà còn bổ sung ROUGE và chrF để đánh giá nhiều khía cạnh của câu sinh ra.

4.3.1 Ý nghĩa của từng nhóm metric

Bảng 4.7 Ý nghĩa của các metric dùng cho mô-đun ghép câu tiếng Việt

Metric	Ý nghĩa	Cách diễn giải
Test Loss	Sai số trung bình khi dự đoán token trên tập test.	Càng thấp càng tốt.
Perplexity	Độ khó dự đoán chuỗi đích của mô hình.	Càng gần 1 càng tốt.
Exact Match	Tỉ lệ câu dự đoán trùng hoàn toàn với câu chuẩn.	Phản ánh độ đúng tuyệt đối.
BLEU	Độ trùng n-gram giữa câu sinh ra và câu tham chiếu.	Phản ánh mức gần giống ở cấp cụm từ.
ROUGE-L	Độ dài chuỗi con chung dài nhất giữa hai câu.	Hữu ích khi quan tâm cấu trúc câu.
chrF	Độ giống nhau ở cấp ký tự.	Nhạy với dấu tiếng Việt và lỗi chính tả nhỏ.
Token F1	Độ trùng token giữa dự đoán và tham chiếu.	Cân bằng precision và recall theo từ.
Similarity Ratio	Độ tương đồng chuỗi ở mức tổng quát.	Hỗ trợ nhìn nhanh mức độ gần giống.

Ý nghĩa các nhóm metric đánh giá sinh văn bản



Các metric được dùng kết hợp để đánh giá cả độ khó dự đoán và chất lượng câu sinh ra.

Hình 4.1 Nhóm ý nghĩa của các metric đánh giá sinh văn bản

4.3.2 Kết quả thực nghiệm trên tập test

Theo báo cáo kỹ thuật của mô-đun VSL_Vietnamese_NLP, BARTpho được đánh giá trên 3.276 mẫu của tập test và cho kết quả như Bảng 4.8. Các số liệu này được lấy trực tiếp từ thư mục output của project: VSL_Vietnamese_NLP/outputs/reports/bartpho_

sentence_reports/. Trong đó, bartpho_sentence_metrics_full.json lưu metric tổng hợp, bartpho_sentence_predictions_full.csv lưu toàn bộ dự đoán trên tập test và bartpho_sentence_sample_predictions_full.txt lưu các mẫu dự đoán dễ đọc. Các giá trị metric này cũng trùng với phần đánh giá trong tài liệu vs1.pdf, mục kết quả BARTpho trên tập test của module ghép gloss thành câu tiếng Việt.

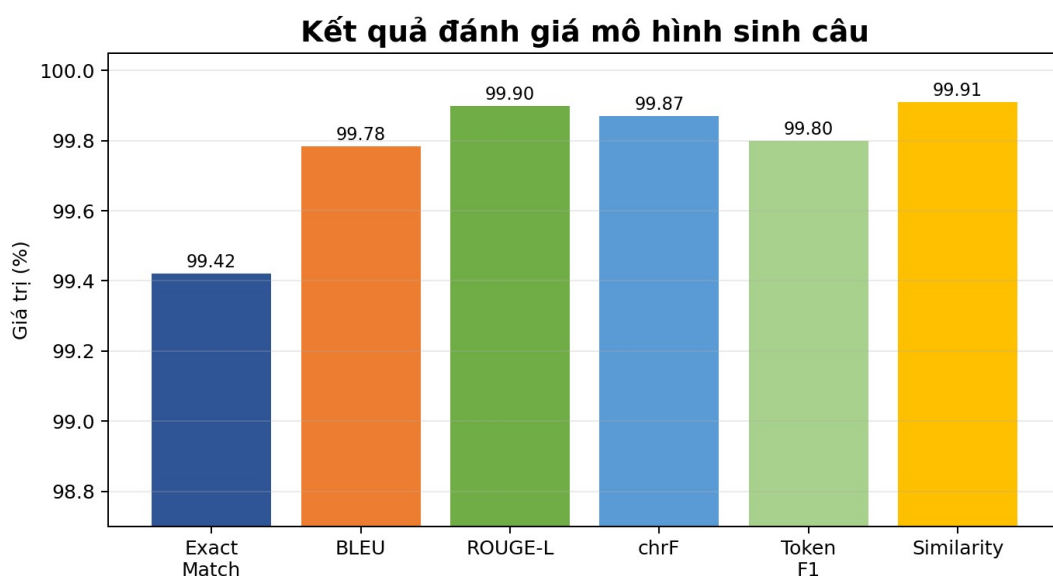
Bảng 4.8 Kết quả đánh giá mô-đun ghép câu tiếng Việt trên tập test

Metric	Giá trị
Số mẫu test	3,276
Test Loss	0,0040
Perplexity	1,0040
Exact Match	0,9942
BLEU	99,7848
ROUGE-L	0,9990
chrF	99,8693
Token F1	0,9980
Similarity Ratio	0,9991
Exact Match đúng/sai	3.257 / 19

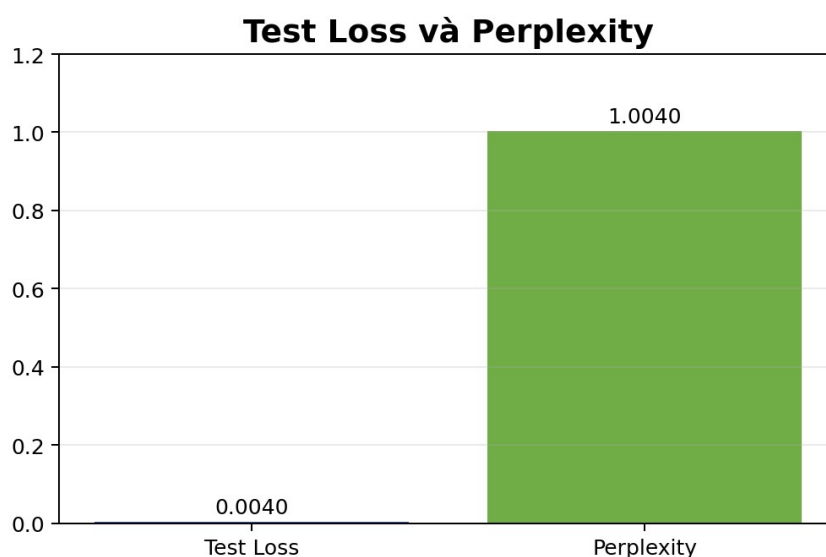
```
Colab - Evaluation Output

KẾT QUẢ ĐÁNH GIÁ FULL TEST SET
num_test_samples: 3276
test_loss: 0.0040
test_perplexity: 1.0040
exact_match: 0.9942
bleu: 99.7848
rouge_l: 0.9990
chrf: 99.8693
token_f1: 0.9980
similarity_ratio: 0.9991
Saved metrics: outputs/reports/bartpho_sentence_metrics_full.json
```

Hình 4.2 Kết quả đánh giá tự động trên toàn bộ tập test



Hình 4.3 Biểu đồ các metric đánh giá BARTpho trên tập test



Hình 4.4 Biểu đồ Test Loss và Perplexity của BARTpho

4.3.3 Phân tích kết quả

Các con số trên cho thấy mô hình hoạt động rất tốt trên bộ test hiện tại. Có thể rút ra một số nhận xét chính:

- **Exact Match = 0,9942** cho thấy gần như toàn bộ câu dự đoán trùng hoàn toàn với câu chuẩn. Với một tác vụ có miền dữ liệu giao tiếp tương đối hẹp, đây là tín hiệu rất mạnh cho thấy mô hình đã học tốt cấu trúc ánh xạ từ gloss sang câu.
- **BLEU = 99,7848** và **ROUGE-L = 0,9990** cho thấy câu sinh ra gần như trùng khớp

hoàn toàn với câu tham chiếu cả ở mức n-gram lẫn cấu trúc chuỗi.

- **chrF = 99,8693** đặc biệt quan trọng với tiếng Việt vì chứng tỏ mô hình giữ được dấu và hình thức ký tự rất tốt.
- **Perplexity = 1,0040** cho thấy mô hình gần như không còn nhiều bất định trên tập test, phù hợp với việc dữ liệu đầu ra có cấu trúc khá ổn định.

Bảng 4.9 trích một số mẫu chưa khớp hoàn toàn từ file dự đoán đầy đủ. Các lỗi này giúp nhìn rõ hơn bản chất sai sót còn lại: đa số là lỗi dấu, nhầm từ gần âm, mất một thành phần ngắn hoặc nhầm mẫu chào hỏi.

Bảng 4.9 Một số mẫu lỗi của mô-đun ghép câu tiếng Việt

Gloss	Câu chuẩn	Câu dự đoán
tam_biet	Tạm biệt.	Tối nay.
xin_loi	Tôi xin lỗi.	Xin chào.
cam_on	Cảm ơn bạn.	Cảm ơn.
me_di_tau_hoa	Mẹ đi tàu hỏa.	Mẹ đi tàu.
ban_can_giay	Bạn cần giấy.	Bạn cần giày.
ban_dong_y	Bạn đồng ý.	Bàn đồng ý.

Nếu chỉ nhìn vào con số, đây là một kết quả gần mức bão hòa. Tuy nhiên, cần đọc đúng bối cảnh của bộ dữ liệu. Theo mô tả trong báo cáo gốc, *VSL Vietnamese NLP Dataset v4 Mega* là một *silver dataset* sinh từ rule và template ở nhiều ngữ cảnh khác nhau. Vì vậy, metric cao phản ánh rất tốt khả năng học lại mẫu câu trong miền dữ liệu đã thiết kế, nhưng chưa đủ để khẳng định mô hình sẽ đạt chất lượng tương đương trên dữ liệu giao tiếp tự do ngoài miền.

4.3.4 Đánh giá định tính và rủi ro diễn giải

Với một hệ thống ứng dụng, cần bổ sung thêm đánh giá định tính bên cạnh metric tự động. Ba tiêu chí nên dùng khi xem mẫu dự đoán gồm:

- **Đúng nghĩa:** câu có giữ đúng nhu cầu hoặc trạng thái được biểu diễn bởi chuỗi gloss không;
- **Đúng ngữ pháp:** câu có đầy đủ thành phần và đúng trật tự tiếng Việt không;
- **Độ tự nhiên:** câu có giống cách người thật thường nói trong ngữ cảnh tương ứng không.

Trong trường hợp của project này, metric rất cao là một tín hiệu tốt để chứng minh mô-đun NLP hoạt động ổn định và có thể tích hợp vào pipeline chung. Tuy nhiên, khi viết kết luận, nên nhấn mạnh rằng đây là chất lượng đạt được trên bộ dữ liệu có cấu trúc rõ ràng; để khẳng định năng lực tổng quát hóa mạnh hơn, cần thêm tập kiểm thử người thật hoặc dữ liệu gloss được sinh từ tầng nhận diện ký hiệu trong điều kiện thực tế.

4.4 Đánh giá mô hình dịch Việt - Lào

Khác với mô-đun ghép câu tiếng Việt, mô-đun dịch Việt - Lào là một bài toán dịch máy đa ngôn ngữ. Vì vậy, việc đánh giá cần bám theo các metric chuẩn của machine translation, đồng thời phải đi kèm các ví dụ định tính để tránh hiểu sai các con số.

4.4.1 Các metric nên sử dụng

Ba metric tự động quan trọng nhất cho cặp Việt - Lào trong đồ án này là:

- **BLEU**: metric cổ điển của Papineni và cộng sự, dựa trên độ trùng *modified n-gram precision*. BLEU phù hợp để so sánh tương đối giữa các lần huấn luyện hoặc giữa các cấu hình mô hình.
- **chrF**: metric của Popović, dựa trên F-score của *character n-gram precision/recall*. Với các ngôn ngữ có hệ chữ viết riêng hoặc dễ phát sinh khác biệt hình thức bề mặt, chrF thường phản ánh tốt hơn BLEU ở mức ký tự.
- **TER**: metric của Snover và cộng sự, đo số phép chỉnh sửa tối thiểu cần để biến câu dự đoán thành câu tham chiếu. Khác với BLEU và chrF, TER là metric *càng thấp càng tốt*.

Ngoài ba metric trên, đối với một hệ thống hỗ trợ giao tiếp, nên bổ sung:

- **đánh giá thủ công về adequacy** (mức bảo toàn ý nghĩa);
- **đánh giá thủ công về fluency** (độ tự nhiên của câu tiếng Lào);
- **kiểm tra lỗi tên riêng, số, địa danh và câu mệnh lệnh**, vì đây là các trường hợp rất dễ ảnh hưởng đến khả năng dùng trong thực tế.

Bảng 4.10 Ý nghĩa của các metric đánh giá mô-đun dịch Việt - Lào

Metric	Ý nghĩa	Cách diễn giải
BLEU	Độ trùng n-gram giữa câu dịch dự đoán và câu tham chiếu.	Càng cao càng tốt; phù hợp để so sánh các cấu hình mô hình.
chrF	Độ giống nhau ở cấp ký tự và recall/precision.	Càng cao càng tốt; hữu ích với ngôn ngữ ít tài nguyên hoặc khác hệ chữ.
TER	Số phép sửa tối thiểu để đưa dự đoán về câu chuẩn.	Càng thấp càng tốt; càng ít phải hậu biên tập.
Adequacy	Mức độ bảo toàn ý nghĩa so với câu nguồn.	Nên chấm thủ công theo thang 1–5.
Fluency	Độ tự nhiên của câu đích đối với người đọc bản ngữ.	Nên chấm thủ công theo thang 1–5.

4.4.2 Giao thức đánh giá nên áp dụng cho project

Với dữ liệu hiện có trong repository, đánh giá chuẩn cho mô-đun dịch nên được thực hiện trên đúng ba tập dữ liệu đã chia sẵn:

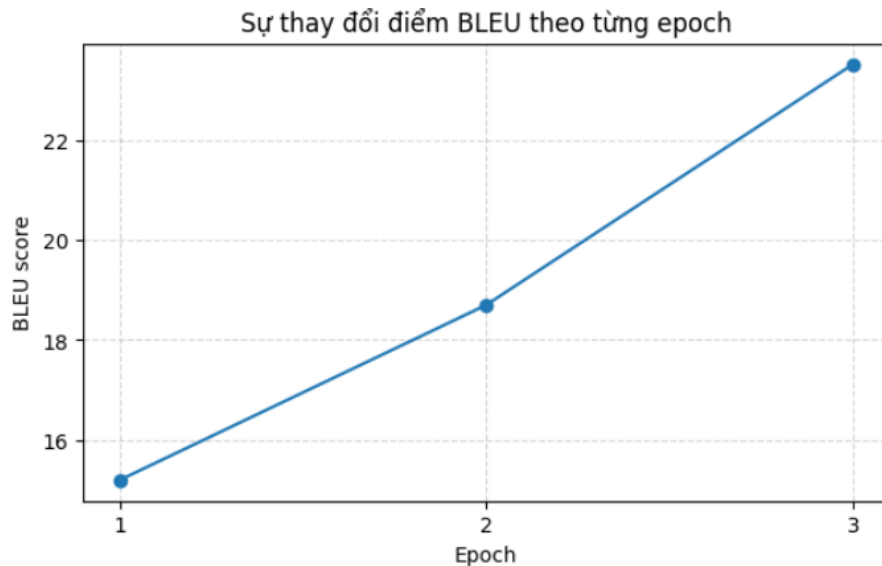
- train: 90.000 cặp câu;
- validation: 5.000 cặp câu;
- test: 5.000 cặp câu.

Trên tập test, quy trình nên gồm các bước:

1. sinh câu tiếng Lào cho toàn bộ 5.000 câu nguồn tiếng Việt bằng chính cấu hình inference đang dùng trong `translate.py`;
2. so sánh câu dự đoán với câu tham chiếu tiếng Lào bằng BLEU, chrF và TER;
3. chọn khoảng 30 đến 50 câu đại diện ở các nhóm ngắn, dài, có tên riêng và có từ chuyên biệt để đánh giá định tính;
4. đối chiếu thêm lỗi lan truyền từ mô-đun ghép câu nếu đánh giá end-to-end.

Bảng 4.11 Kết quả đánh giá định lượng mô-đun dịch Việt – Lào trích từ `vilo.pdf`

Chỉ số	Giá trị	Ý nghĩa
Train loss	4,67	Mức độ lỗi trong quá trình huấn luyện.
Validation loss	1,06	Mức độ lỗi trên tập validation.
BLEU trên tập kiểm thử	23,57	Mức độ giống giữa bản dịch của mô hình và bản dịch chuẩn.



Hình 4.5 Sự thay đổi BLEU của mô-đun dịch Việt – Lào theo từng epoch

4.4.3 Nhận xét dựa trên trạng thái repository hiện tại

Tại thời điểm viết báo cáo, snapshot mã nguồn hiện có:

- bộ dữ liệu train/validation/test đã chia sẵn;
- adapter LoRA đã huấn luyện;
- script suy luận cho từng câu đầu vào.
- tài liệu `vilo.pdf` ghi nhận Train loss = 4,67, Validation loss = 1,06 và BLEU = 23,57.

Kết quả BLEU = 23,57 cho thấy mô hình đã học được khả năng dịch tương đối cho cặp Việt – Lào, đặc biệt trong bối cảnh dữ liệu song ngữ ít tài nguyên hơn các cặp ngôn ngữ phổ biến. Tuy nhiên, mức BLEU này chưa phải chất lượng cao tuyệt đối; khi triển khai thực tế vẫn cần đánh giá thêm bằng ví dụ định tính và kiểm tra lỗi trên các câu dài, tên riêng, số và câu khẩu ngữ.

Trong workspace hiện tại chưa thấy số liệu chrF và TER tương ứng với cùng lần đánh giá. Vì vậy, cách trình bày trung thực nhất trong báo cáo là:

- mô tả rõ mô hình đang dùng là NLLB-200 Distilled 600M + LoRA;
- công bố các số liệu có nguồn trong `vilo.pdf`: Train loss, Validation loss và BLEU;
- giữ chrF và TER ở mức giao thức đề xuất, chưa công bố số nếu chưa tái lập được phép đo;

- tránh bị thêm số liệu không có khả năng truy vết trong workspace.

Điểm này không phải là thiếu sót về học thuật, miễn là báo cáo ghi rõ trạng thái hiện tại và không trình bày các con số không có khả năng truy vết.

4.4.4 Đánh giá định tính nên nhấn mạnh điều gì

Ngay cả khi đã có BLEU, chrF và TER, phần dịch Việt – Lào vẫn cần thêm một bảng ví dụ để người đọc thấy những kiểu lỗi quan trọng. Các nhóm ví dụ ưu tiên gồm:

- câu nhu cầu cơ bản ngắn, ví dụ *Tôi muốn uống nước*;
- câu có tên riêng như bệnh viện, địa danh, tên người;
- câu dài hơn có mệnh đề phụ, ví dụ câu mô tả tình trạng sức khỏe hoặc yêu cầu trợ giúp;
- câu chứa số, thời gian hoặc thực thể viết hoa.

Quan sát nhanh trên dữ liệu song ngữ hiện có cho thấy các điểm rủi ro lớn nhất của mô-đun dịch là:

- tên riêng và từ phiên âm dễ bị sai hình thức;
- câu dài dễ lặp lại hoặc diễn đạt rườm rà;
- dữ liệu huấn luyện nhiều miền có thể làm mô hình không đồng đều chất lượng giữa các kiểu câu.

4.4.5 Ý nghĩa của đánh giá đối với toàn hệ thống

Mô-đun dịch là tầng cuối cùng trước khi thông tin đến tay người dùng đích. Vì vậy, nếu mô-đun này dịch sai nghĩa, toàn bộ giá trị của pipeline trước đó có thể bị giảm mạnh. Đó là lý do phần đánh giá dịch Việt - Lào cần được xem như một bước kiểm chứng độc lập và đồng thời là một bước kiểm chứng cho chất lượng của cả hệ thống end-to-end.

4.5 Đánh giá tích hợp toàn hệ thống

Ngoài việc đánh giá độc lập từng mô hình, đề tài cần đánh giá pipeline tổng thể để phản ánh đúng giá trị ứng dụng thực tế. Đánh giá end-to-end không chỉ quan tâm từng nhãn ký hiệu có đúng hay không, mà còn xem câu tiếng Việt và câu tiếng Lào cuối cùng có truyền đạt đúng ý định ban đầu của người dùng hay không.

4.5.1 Nội dung nên đánh giá

- Độ đúng của đầu ra cuối cùng từ video đến câu tiếng Việt và tiếng Lào.
- Tốc độ phản hồi của toàn pipeline.
- Tính ổn định của hệ thống khi chạy webcam liên tục.
- Ảnh hưởng của lỗi mô-đun trước lên mô-đun sau.

Bảng 4.12 Đánh giá định tính pipeline end-to-end

Kịch bản thử nghiệm	Kết quả	Nhận xét
Người dùng ký hiệu câu ngắn, gồm 1–2 token	Khả thi trong demo	Khi token nhận diện đúng và có rule/fallback phù hợp, hệ thống sinh được câu tiếng Việt rõ nghĩa và có thể chuyển sang tầng dịch.
Người dùng ký hiệu nhiều token liên tiếp	Cần thao tác xác nhận	Chất lượng phụ thuộc mạnh vào việc chọn đúng token vào buffer; sai một token có thể làm câu tiếng Việt lệch nghĩa.
Điều kiện ánh sáng yếu hoặc tay ra khỏi khung hình	Không ổn định	Tầng nhận diện là điểm nghẽn chính; khi landmarks mất ổn định, toàn bộ pipeline phía sau bị ảnh hưởng.
Chạy không tải mô hình dịch	Ổn định hơn	Tùy chọn -no-translation phù hợp để kiểm thử nhanh hai tầng đầu hoặc chạy trên máy hạn chế tài nguyên.
Chạy đầy đủ có dịch Việt – Lào	Phụ thuộc tài nguyên máy	Lần chạy đầu có thể cần tải mô hình nền NLLB; độ trễ cao hơn so với chỉ sinh câu tiếng Việt.

Kết quả tích hợp cho thấy điểm quan trọng nhất của hệ thống là khả năng nói được ba mô-đun vốn có bản chất khác nhau: thị giác, sinh câu tiếng Việt và dịch máy. Tuy nhiên, đây vẫn là nguyên mẫu có người vận hành hỗ trợ xác nhận token. Để đạt mức sản phẩm hoàn chỉnh, hệ thống cần thêm cơ chế tự động tách câu, lọc dự đoán nhiễu, đánh giá ngưỡng tin cậy và kiểm thử trên nhiều người dùng thật.

4.6 Kết luận đề tài

Đề tài đã hiện thực hóa được một pipeline đa mô-đun cho bài toán hỗ trợ giao tiếp bằng ngôn ngữ ký hiệu, gồm ba tầng chính: nhận diện ký hiệu từ webcam, khôi phục câu tiếng Việt từ chuỗi gloss/token và dịch câu tiếng Việt sang tiếng Lào. Cách tiếp cận này chứng minh tính khả thi của việc kết hợp thị giác máy tính, xử lý ngôn ngữ tự nhiên và dịch máy trong cùng một hệ thống ứng dụng.

Ở tầng nhận diện, hệ thống tích hợp thành công mô hình TFLite pre-trained từ Kaggle ASL Signs với MediaPipe Holistic, sử dụng toàn bộ 543 landmarks và cơ chế cửa sổ trượt 16–60 frame. Nhóm đã bổ sung công cụ đánh giá để tính Top-k Accuracy, Precision, Recall, F1 và confusion matrix khi có tập kiểm thử gán nhãn; trong snapshot hiện tại chỉ công bố latency TFLite vì chưa có test manifest nhận diện riêng. Ở tầng tiếng Việt, hệ thống sử dụng kiến trúc lai gồm rule, BARTpho và fallback để chuyển chuỗi token thành câu tiếng Việt hoàn chỉnh. Kết quả đánh giá trên tập test của mô-đun tiếng Việt cho thấy các metric đạt mức rất cao trong miền dữ liệu được thiết kế. Ở tầng dịch, hệ thống tích hợp NLLB-200 Distilled 600M với adapter LoRA cho cặp Việt – Lào; tài liệu vil0.pdf ghi nhận Train loss = 4,67, Validation loss = 1,06 và BLEU = 23,57.

Về mặt học thuật, đề tài giúp hệ thống hóa quy trình xây dựng một ứng dụng AI nhiều tầng, trong đó mỗi mô hình có vai trò, dữ liệu và metric đánh giá riêng. Về mặt ứng dụng, project tạo được nguyên mẫu có thể chạy demo từ webcam, có cơ chế buffer token và có khả năng bật/tắt tầng dịch tùy theo tài nguyên máy. Những kết quả này đáp ứng mục tiêu ban đầu của đề tài ở mức nguyên mẫu nghiên cứu và là nền tảng để mở rộng thành hệ thống thực tế hơn trong tương lai.

4.7 Hạn chế hiện tại

Mặc dù hệ thống đã nổi được ba mô-đun chính, phiên bản hiện tại vẫn còn một số hạn chế cần được nhìn nhận thẳng thắn.

- Mô hình nhận diện đang dùng bộ ASL-250 pre-trained, chưa phải mô hình VSL thuần được huấn luyện trên dữ liệu người dùng Việt Nam.
- Chất lượng nhận diện còn nhạy với ánh sáng, góc quay, khoảng cách tới camera, che khuất tay và tốc độ ký hiệu.
- Pipeline hiện nhận diện theo token/gloss có thao tác xác nhận, chưa xử lý hoàn toàn tự động câu ký hiệu liên tục dài.
- Sai số ở tầng nhận diện có thể lan sang tầng khôi phục câu và tầng dịch, đặc biệt khi một token quan trọng bị nhận sai.

- Metric của mô-đun tiếng Việt rất cao trên bộ dữ liệu có cấu trúc/template, nhưng chưa đủ để khẳng định chất lượng tương đương trên dữ liệu giao tiếp tự do.
- Mô-đun dịch Việt – Lào đã có Train loss, Validation loss và BLEU trong `vilo.pdf`, nhưng chưa có chrF và TER truy vết trong workspace nên cần tái lập thêm nếu muốn đánh giá đầy đủ hơn.
- Hệ thống hiện mới dừng ở mức nguyên mẫu demo, chưa được đóng gói thành ứng dụng desktop/web/mobile hoàn chỉnh.

4.8 Hướng phát triển

Trên cơ sở các hạn chế hiện tại, đề tài có thể được phát triển theo nhiều hướng nhằm nâng cao cả độ chính xác lẫn khả năng ứng dụng thực tế.

- Thu thập hoặc xây dựng bộ dữ liệu VSL thật với nhiều người ký hiệu, nhiều góc quay và nhiều điều kiện ánh sáng để huấn luyện/fine-tune mô hình nhận diện phù hợp hơn với tiếng Việt.
- Xây dựng tập kiểm thử nhận diện có nhãn thật và file manifest path, label để công bố đầy đủ Top-1, Top-3, Top-5 Accuracy, Macro F1 và confusion matrix cho tăng thị giác.
- Mở rộng số lớp ký hiệu và bổ sung cơ chế nhận diện câu ký hiệu liên tục, bao gồm tách đoạn động tác, lọc nhiễu và gom token tự động.
- Thử nghiệm các mô hình chuỗi hoặc video mạnh hơn như Transformer-based video models, Temporal Convolution hoặc Graph Neural Network trên landmarks.
- Cải thiện mô-đun khôi phục câu bằng dữ liệu gloss – câu phong phú hơn, đặc biệt là dữ liệu sinh từ lỗi thật của tầng nhận diện.
- Tái lập đánh giá đầy đủ cho mô-đun dịch Việt – Lào trên validation/test set, lưu log metric và bổ sung đánh giá thủ công bởi người biết tiếng Lào.
- Bổ sung text-to-speech cho tiếng Việt hoặc tiếng Lào để hệ thống hỗ trợ giao tiếp bằng giọng nói.
- Đóng gói thành ứng dụng desktop, web hoặc mobile, kèm giao diện thân thiện hơn và cơ chế cấu hình camera/mô hình.

TÀI LIỆU THAM KHẢO

1. TS. Lê Quốc Uy, *Slide bài giảng học phần Học Sâu*, Khoa Kỹ Thuật Điện Tử 1, Học viện Công nghệ Bưu chính Viễn thông.
2. Camillo Lugaresi, Jiuqiang Tang, Hadon Nash, Chris McClanahan, Esha Uboweja, Michael Hays và cộng sự, *MediaPipe: A Framework for Building Perception Pipelines*, arXiv 2019. URL: <https://arxiv.org/abs/1906.08172>.
3. Google, *MediaPipe Holistic Documentation*. Truy cập tại: https://ai.google.dev/edge/mediapipe/solutions/vision/holistic_landmarker.
4. Dongxu Li, Cristian Rodriguez Opazo, Xin Yu và Hongdong Li, *Word-level Deep Sign Language Recognition from Video: A New Large-scale Dataset and Methods Comparison*, WACV 2020. URL: https://openaccess.thecvf.com/content_WACV_2020/papers/Li_Word-level_Deep_Sign_Language_Recognition_from_Video_A_New_Large-scale_WACV_2020_paper.pdf.
5. Sepp Hochreiter và Jürgen Schmidhuber, *Long Short-Term Memory*, Neural Computation, 1997. URL: <https://direct.mit.edu/neco/article/9/8/1735/6109/Long-Short-Term-Memory>.
6. Mike Schuster và Kuldeep K. Paliwal, *Bidirectional Recurrent Neural Networks*, IEEE Transactions on Signal Processing, 1997. URL: <https://deeplearning.cs.cmu.edu/S22/document/readings/Bidirectional%20Recurrent%20Neural%20Networks.pdf>.
7. Mike Lewis, Yinhan Liu, Naman Goyal, Marjan Ghazvininejad, Abdelrahman Mohamed, Omer Levy, Ves Stoyanov và Luke Zettlemoyer, *BART: Denoising Sequence-to-Sequence Pre-training for Natural Language Generation, Translation, and Comprehension*, ACL 2020. URL: <https://arxiv.org/abs/1910.13461>.
8. Nguyen Luong Tran, Duong Minh Le và Dat Quoc Nguyen, *BARTpho: Pre-trained Sequence-to-Sequence Models for Vietnamese*, INTERSPEECH 2022. URL: <https://arxiv.org/abs/2109.09701>.
9. NLLB Team, Marta R. Costa-jussà, James Cross, Onur Çelebi và cộng sự, *No Language Left Behind: Scaling Human-Centered Machine Translation*, 2022. URL: <https://arxiv.org/abs/2207.04672>.
10. Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yanzhi Li, Shean Wang, Lu Wang và Weizhu Chen, *LoRA: Low-Rank Adaptation of Large Language Models*, 2021. URL: <https://arxiv.org/abs/2106.09685>.

11. Hugging Face, *NLLB Model Documentation*. Truy cập tại: https://huggingface.co/docs/transformers/model_doc/nllb.
12. Kishore Papineni, Salim Roukos, Todd Ward và Wei-Jing Zhu, *BLEU: a Method for Automatic Evaluation of Machine Translation*, ACL 2002. URL: <https://aclanthology.org/www.mt-archive.info/ACL-2002-Papineni.pdf>.
13. Chin-Yew Lin, *ROUGE: A Package for Automatic Evaluation of Summaries*, Workshop on Text Summarization Branches Out, 2004. URL: <https://aclanthology.org/W04-1013/>.
14. Maja Popović, *chrF: character n-gram F-score for automatic MT evaluation*, WMT 2015. URL: <https://aclanthology.org/W15-3049/>.
15. Matthew Snover, Bonnie Dorr, Rich Schwartz, Linnea Micciulla và John Makhoul, *A Study of Translation Edit Rate with Targeted Human Annotation*, AMTA 2006. URL: <https://aclanthology.org/2006.amta-papers.25/>.
16. Mahya Fayyazsanavi, Ruwanthi Attanayake và Richard Bowden, *Gloss2Text: Sign Language Gloss Translation using LLMs and Semantically Aware Label Smoothing*, Findings of EMNLP 2024. URL: <https://aclanthology.org/2024.findings-emnlp.947/>.
17. Kaggle, *Google – Isolated Sign Language Recognition Competition*. Truy cập tại: <https://www.kaggle.com/competitions/asl-signs>.
18. hoyso48, *1st Place Solution – Inference*, Kaggle ASL Signs Competition. Truy cập tại: <https://www.kaggle.com/code/hoyso48/1st-place-solution-inference>.
19. Google, *TensorFlow Lite Documentation*. Truy cập tại: <https://www.tensorflow.org/lite>.